



projet de fin d'année

présenté par :

NESSRINE LAFHAL

Fusion Innovante de l'Intelligence Artificielle et des Technologies
Mobiles pour la Traduction Automatique de la Langue des Signes en
Texte et Message Vocal Vers une Communication Inclusive

Soutenu le 09/09/2024

Composition du jury :

Encadrant académique : Nidal Lamghari

Encadrant d'entreprise : Dr. Kayyali

Responsable : DARGHAM Abdelmajid

Examineur : ABOUELKHIR Imane

تعتبر لغة الإشارة وسيلة التواصل الأساسية للأشخاص الصم والبكم، لكنها لا تُفهم دائمًا من قبل الأفراد السامعين، يهدف مشروعنا إلى تطوير تطبيق موبايل يتيح ترجمة لغة الإشارة إلى نص ورسالة صوتية باستخدام تقنيات الذكاء الاصطناعي المتقدمة. من خلال هذا المشروع، قمنا بتصميم تطبيق متعدد المنصات باستخدام فلاتر، قادر على تحويل إشارات اليد إلى نص مكتوب، ثم تحويل هذا النص إلى رسالة صوتية.

في هذا المشروع، ندمج تقنيات الذكاء الاصطناعي مثل الشبكات العصبية التلافيفية (CNN)، وشبكات الذاكرة طويلة الأمد (LSTM)، وغيرها لتحسين دقة ترجمة إشارات اليد إلى نصوص. يتم تحويل النصوص الناتجة إلى رسائل صوتية لتسهيل التواصل بين الأفراد الذين لا يجيدون لغة الإشارة.

يساهم هذا المشروع في تحسين التواصل بين الأشخاص الصم والبكم والأشخاص السامعين، ويعزز الإدماج الاجتماعي من خلال توفير وسيلة تواصل فعالة وشاملة. كما يوفر التطبيق منصة قوية لدعم تقنيات الذكاء الاصطناعي في التطبيقات العملية لتحسين جودة حياة الأفراد

Abstract

Dans un monde où la communication est nécessaire, les personnes sourdes et malentendantes sont souvent confrontées à des obstacles lorsqu'elles interagissent avec des gens qui n'ont pas de compétences en langue des signes. Pour surmonter ces obstacles, notre projet se concentre sur le développement d'une application mobile innovante utilisant Flutter, qui traduit les gestes du langage des signes en textes écrits et messages vocaux.

Le projet intègre des modèles d'apprentissage profond, tels que les réseaux de neurones synthétiques (CNN) et les réseaux de neurones à longue portée (LSTM), pour reconnaître les gestes. Ces modèles sont couplés à la compilation RandomForest pour améliorer la précision de la traduction. L'application permet aux utilisateurs de communiquer en toute transparence avec ceux qui ne connaissent pas le langage des signes en convertissant les gestes capturés par la caméra en messages texte et audio lisibles, facilitant ainsi l'inclusion sociale et l'accès à la communication.

Le système utilise Flutter pour l'interface utilisateur sur toute la plateforme, assurant ainsi la compatibilité entre différents appareils mobiles. Cette solution innovante vise à améliorer la qualité de vie des sourds et malentendants en rendant le langage gestuel plus accessible et compréhensible pour tous.

Résumé

Dans un monde où la communication est essentielle, les personnes sourdes et malentendantes sont souvent confrontées à des obstacles lorsqu'elles interagissent avec des personnes qui n'ont pas de compétences en langue des signes. Ce projet vise à développer une application mobile innovante, utilisant le cadre de travail Flutter, pour surmonter ces défis. L'application traduit les gestes en langue des signes en messages écrits et audio, facilitant la communication entre les personnes sourdes et celles qui ne connaissent pas le langage des signes.

Le projet intègre des modèles avancés d'apprentissage profond, y compris les réseaux de neurones synthétiques (CNN) et les réseaux de neurones à long terme (LSTM), pour identifier avec précision les gestes. Ces modèles sont combinés avec le classificateur RandomForest pour améliorer la précision de la traduction. En convertissant les gestes capturés par la caméra en texte et en son, l'application permet une interaction fluide et naturelle, contribuant à améliorer l'inclusion sociale et l'accès à la communication.

En utilisant Flutter pour l'interface utilisateur multi-plateforme, l'application assure la compatibilité entre différents appareils mobiles. Cette solution innovante vise à améliorer la qualité de vie des sourds et malentendants en rendant le langage gestuel plus accessible et compréhensible pour tous.

Remerciements

Je tiens à exprimer ma profonde gratitude à ma superviseure, la professeure Nidal Lamghari, pour son soutien précieux et ses conseils éclairés tout au long de ce projet. Ses conseils techniques ont été essentiels pour développer cette application et comprendre les défis techniques rencontrés. Son expertise et sa disponibilité ont grandement contribué au succès de ce travail.

Je remercie également le directeur de mon entreprise, Dr. Kayyali, pour son aide et sa précieuse participation au projet. Ses recommandations pratiques et son soutien continu ont été utiles pour la mise en application réussie des concepts dans la vie réelle. Sa compréhension des besoins du terrain et son appui logistique ont permis d'atteindre les objectifs fixés.

Je vous suis très reconnaissant pour votre engagement, votre patience et votre contribution essentielle à la réalisation de ce projet.

Dédicace

Nous dédions ce projet à tous ceux qui ont joué un rôle déterminant dans notre parcours académique et personnel, qui ont rendu possible ce travail grâce à leur soutien et à leurs encouragements.

Pour nos familles, les piliers de notre existence, qui ont tant sacrifié pour nous voir réussir. Cela nous a appris la valeur du travail acharné et de la persévérance. Votre amour et votre soutien indéfectible ont été notre sanctuaire en temps de doute et notre force en temps difficiles. Nous vous consacrons ce succès en gage de notre gratitude éternelle.

ma superviseure, Mm. nidal lamghari, dont les conseils et l'orientation spécialisée ont été essentiels au succès de ce projet. Votre engagement et votre disponibilité ont grandement contribué à notre compréhension des défis techniques auxquels nous sommes confrontés.

À notre superviseur, Dr. Kayyali, pour son aide et sa précieuse participation dans la mise en œuvre pratique des concepts. Votre soutien continu et vos recommandations pratiques ont été déterminants pour le succès de cette demande.

Enfin, nous dédions ce projet à tous ceux qui croient en l'importance de l'inclusion et de la communication. Ce travail est le résultat de notre coopération et de notre engagement et symbolise notre désir de contribuer positivement à la société avec les compétences et les connaissances acquises.

Ce projet n'est pas seulement pour nous, c'est aussi pour vous. Merci d'être une partie intégrante de cette grande aventure

Table des matières

Liste des tableaux	10
Table des figures	11
Liste des acronymes	14
Introduction générale	16
1 Introduction	18
1.1 Contexte du Projet	18
1.2 Objectif du Stage	19
1.3 Compétences acquises	20
1.4 Présentation globale de 4D Business Consulting	21
1.4.1 Introduction	21
1.4.2 Nos Services	21
1.4.2.1 Gestion des Services Technologiques	21
1.4.2.2 Externalisation	21
1.4.2.3 Stratégies Personnalisées	21
1.4.3 Notre Approche	22
1.4.3.1 Collaboration avec les Clients	22
1.4.3.2 Résultats et Performances	22

1.4.3.3	Innovation et Développement Stratégique	22
2	Étude et Recherche	23
2.1	Base de Données de la Langue des Signes	23
2.1.1	Structure de la Base de Données	23
2.1.2	Exemple de Code pour la Capture des Données	24
2.1.3	Actions Détectées	32
2.2	Technologies Utilisées	33
2.2.1	Vision par Ordinateur	33
2.2.2	Flutter	34
2.2.3	Dart	35
2.2.4	Flask	35
2.2.5	Émulateur	35
2.2.6	Bibliothèques et Packages	36
2.2.7	Environnement de Développement	36
3	Structuration et Modélisation	37
3.1	Diagramme de Cas d'Utilisation	37
3.2	Diagramme de Séquence	38
3.3	Diagramme de Classes	39
3.4	Diagramme d'Activité	40
3.5	Diagramme d'État de Transition	41
3.6	Diagramme de Flux des Données	42
4	Partie Intelligence Artificielle	43
4.1	Développement et Test des Algorithmes	43
4.1.1	Réseaux de Neurones Convolutifs (CNN 3D)	45

4.1.2	Réseaux de Neurones Universels (RNU)	47
4.1.3	Long Short-Term Memory (LSTM)	49
4.1.4	Random Forest	51
4.1.5	Machines à Vecteurs de Support (SVM)	52
4.1.6	Combinaisons de Modèles	55
4.1.6.1	LSTM + CNN	55
4.1.6.2	LSTM + Random Forest	57
4.1.6.3	LSTM + CNN + Random Forest	60
4.2	Comparaison et Sélection du Modèle	61
4.2.1	Comparaison du Modèle	61
4.2.2	Sélection du Modèle	66
5	Partie Développement	69
5.1	Architecture de l'Application Mobile Flutter	69
5.1.1	Introduction au Langage Dart et Projet Flutter	69
5.1.1.1	Structure du Projet Flutter	70
5.1.1.2	Conception des Interfaces Utilisateur	71
5.1.1.3	Gestion des États et Navigation	73
5.2	Développement Backend avec Flask	74
5.2.1	Structure de l'Application Flask	74
5.2.2	Création des API pour la Communication avec Flutter	74
5.2.3	Chargement et Utilisation du Modèle CNN+LSTM	75
5.2.4	Gestion des Requêtes et Réponses	75
5.2.5	Intégration du Modèle de Machine Learning	76
5.2.5.1	Importation du Modèle CNN + LSTM	76

5.2.5.2	Prétraitement des Données d'Entrée	76
5.2.5.3	Génération des Prédictions et Traitement des Résultats	77
5.3	Connexion entre Flutter et Flask	78
5.3.1	Configuration des Requêtes HTTP dans Flutter	78
5.3.2	Traitement des Réponses de l'API Flask dans Flutter	79
5.3.3	Traduction en message vocal	79
5.3.4	Gestion des Erreurs et Débogage	80
5.4	Tests Fonctionnels	81
5.4.1	Tests Fonctionnels de l'Application Mobile	81
5.4.2	Tests de l'API Flask et du Modèle	81
5.4.3	Validation de l'Intégration et Performances	82
5.5	Déploiement et Maintenance	82
5.5.1	Déploiement de l'Application Flutter	82
5.5.2	Déploiement du Backend Flask	83
5.5.3	Stratégies de Maintenance et Mise à Jour	83
5.6	Conclusion	84
5.6.1	Résumé du Fonctionnement de l'Application	84
5.6.2	Évaluation des Résultats et Performances	88
5.6.3	Perspectives et Améliorations Futures	90

Conclusion générale	91
----------------------------	-----------

Liste des tableaux

4.1 Comparaison Des Modèles 62

Table des figures

2.1	Les bibliothèques utilisées	24
2.2	Points clés utilisant MP Holistic	24
2.3	fonction draw styled landmarks	25
2.4	fonction mediapipe detection	25
2.5	fonction draw landmarks	25
2.6	capture un flux vidéo en temps réel	26
2.7	Extraction et Normalisation des Points de Repère	27
2.8	enregistre les images sous forme de tableaux NumPy	27
2.9	collecter les données	28
2.10	geste 1	29
2.11	geste 2	29
2.12	geste 3	30
2.13	Preparation des données	31
2.14	Base de données	32
2.15	Les 30 Séquences d'une action	32
2.16	Les 30 fichier de format .npy	33
2.17	Capture de geste	34
2.18	Émulateur Android	36

3.1	Diagramme de Cas d'Utilisation	37
3.2	Diagramme de Séquence	38
3.3	Diagramme de Classes	39
3.4	Diagramme d'Activité	40
3.5	Diagramme d'État de Transition	41
3.6	Diagramme de Flux de Données	42
4.1	CNN-3D	45
4.2	Confusion Matrix CNN-3D	46
4.3	Performance de CNN-3D	46
4.4	RNU Training	47
4.5	Confusion Matrix RNU	48
4.6	Confusion Matrix RNU	48
4.7	Entraînement de LSTM	50
4.8	Confusion Matrix LSTM	50
4.9	Confusion Matrix RandomForest	52
4.10	Visualisation de Performance RandomForest	52
4.11	Confusion Matrix SVM	54
4.12	Visualisation de Performance SVM	54
4.13	CNN+LSTM Training	55
4.14	Confusion Matrix CNN+LSTM	56
4.15	Visualisation de Performance CNN +LSTM	56
4.16	Confusion Matrix RandomForest+LSTM	59
4.17	Visualisation de Performance de RandomForest +LSTM	59
4.18	Matrice de Confusion CNN+RandomForest+LSTM	61

4.19	Histogramme des Models	64
4.20	Heatmap des performance des Models	64
4.21	Graphiques en barres	65
4.22	Graphiques de lignes	65
5.1	Structure du Projet Flutter	70
5.2	Page d'accueil	72
5.3	Page d'inscription	72
5.4	Menu de Navigation	72
5.5	Page Principale	72
5.6	Intégration du Modèle	76
5.7	Prétraitement des Données d'Entrée	76
5.8	Génération des Prédictions	77
5.9	Configuration des Requêtes HTTP	78
5.10	Traitement des Réponses de l'API	79
5.11	Traduction en Message Vocal	79
5.12	Gestion des Erreurs et Débogage	80
5.13	Prediction avec sucess	80
5.14	Page d'Accueil	84
5.15	Page d'Inscription	85
5.16	Page de Reconnaissance Gestuelle	86
5.17	Menu de Navigation	87
5.18	Test des gestes	88
5.19	Test des gestes	89

Liste d'abréviations

- **CNN** : Convolutional Neural Network (Réseau de Neurones Convolutifs)
- **LSTM** : Long Short-Term Memory
- **RNU** : Réseaux de Neurones Universels
- **SVM** : Support Vector Machine (Machines à Vecteurs de Support)
- **RF** : Random Forest
- **AI** : Artificial Intelligence (Intelligence Artificielle)
- **ML** : Machine Learning (Apprentissage Automatique)
- **DL** : Deep Learning (Apprentissage Profond)
- **NLP** : Natural Language Processing (Traitement du Langage Naturel)
- **SL** : Sign Language (Langue des Signes)
- **HMM** : Hidden Markov Model (Modèle de Markov Caché)
- **ANN** : Artificial Neural Network (Réseau de Neurones Artificiels)
- **F1** : F1 Score
- **TPR** : True Positive Rate (Taux de Vrais Positifs)
- **FPR** : False Positive Rate (Taux de Faux Positifs)
- **TNR** : True Negative Rate (Taux de Vrais Négatifs)
- **FDR** : False Discovery Rate (Taux de Découverte de Faux Positifs)

Introduction générale

La communication est une partie essentielle de notre vie quotidienne et essentielle pour l'échange d'idées, la formation de relations et la participation active dans la société. Cependant, pour les personnes sourdes et malentendantes, les barrières à la communication peuvent être particulièrement restrictives. En fait, bien que la langue des signes soit le principal moyen de communication, elle est souvent mal comprise par les auditeurs, ce qui limite leur capacité à interagir pleinement avec le monde extérieur.

Dans ce contexte, notre projet de développement d'applications mobiles vise à relever ces défis en proposant une solution technologique innovante. L'objectif est de créer une application via une plateforme utilisant le framework Flutter, capable de traduire des gestes du langage des signes en texte écrit puis de convertir ce texte en message audio. Cette application vise à faciliter la communication entre les sourds et malentendants et ceux qui n'ont pas de compétences en langue des signes, améliorant ainsi l'inclusion sociale et l'accessibilité.

Pour atteindre cet objectif, nous avons défini un plan d'action qui comporte plusieurs étapes clés. Une phase de recherche et d'étude sera menée afin de comprendre les fondamentaux du langage des signes et d'explorer les techniques actuelles en reconnaissance gestuelle et intelligence artificielle. Ensuite, nous allons concevoir la structure de l'application, créer des modèles sans fil et des cadres pour l'interface utilisateur.

Le développement de l'application comprendra la mise en œuvre dans Flutter, l'intégration de bibliothèques pour reconnaître les gestes via une caméra de smartphone et la création d'un module de traduction pour convertir les gestes en texte. En outre, l'API sera intégrée pour synthétiser la parole et convertir le texte en messages vocaux.

La partie intelligence artificielle du projet sera centrée sur la collecte et la préparation des données, le développement et le test des algorithmes de machine learning, y compris les réseaux de neurones convolutifs (CNN), les forêts aléatoires (Random Forest), et les réseaux de neurones à long terme (LSTM). Nous comparerons les performances de ces modèles pour sélectionner le plus performant et l'intégrer dans l'application.

Enfin, nous réaliserons des tests et une validation rigoureuse pour garantir la fonctionnalité de l'application et recueillir les retours des utilisateurs.

Ce projet représente une opportunité significative pour améliorer la qualité de vie des personnes sourdes et malentendantes, en leur offrant un outil efficace pour surmonter les barrières de communication et favoriser leur intégration dans la société.

Chapitre 1

Introduction

1.1 Contexte du Projet

Dans un monde de plus en plus interdépendant, la communication est un élément essentiel d'inclusion sociale et professionnelle. Cependant, pour les personnes sourdes et malentendantes, les interactions quotidiennes peuvent présenter des défis importants lorsqu'elles doivent communiquer avec des personnes qui ne parlent pas le langage gestuel. La langue des signes est le principal moyen de communication pour cette société, mais elle reste encore inconnue et parfois mal comprise par ceux qui ne l'utilisent pas. Cette lacune peut entraîner des obstacles importants à la communication, limitant les possibilités d'interaction sociale et d'intégration des sourds et malentendants.

Pour relever ces défis, le projet de développement d'applications mobiles vise à traduire les gestes en langage gestuel en textes écrits et messages vocaux. Grâce au cadre Flutter, ce projet offre une solution technique avancée qui permet de convertir les gestes capturés en temps réel par la caméra du smartphone en texte lisible puis de convertir ce texte en messages vocaux. Cette approche vise non seulement à améliorer la communication entre les personnes utilisant le langage des signes et celles qui ne l'utilisent pas, mais aussi à promouvoir une meilleure inclusion sociale et un accès plus équitable à la communication.

Le projet est mis en œuvre dans un contexte où l'innovation technologique offre des possibilités sans précédent de surmonter les obstacles à la communication et d'améliorer la qualité de vie des personnes handicapées. En intégrant des technologies avancées de reconnaissance gestuelle et d'IA, l'application vise à fournir un outil pratique et efficace qui répond aux besoins spécifiques de cette communauté.

1.2 Objectif du Stage

Cette formation a pour principal objectif de développer une application mobile innovante utilisant le cadre Flutter pour traduire la langue des signes en textes écrits et messages vocaux. Cette application vise à faciliter la communication entre les sourds, les malentendants et ceux qui n'ont pas de compétences en langue des signes, en offrant une solution technologique pratique pour surmonter les barrières de communication.

Les objectifs spécifiques du stage incluent :

Conception et Développement de l'Application : Créer une application mobile multiplateforme en utilisant Flutter, capable de capturer les gestes de la langue des signes à l'aide de la caméra du smartphone et de les traduire en texte écrit. L'application devra également intégrer une fonctionnalité de synthèse vocale pour convertir le texte en messages vocaux.

Intégration des Modèles d'Intelligence Artificielle : Implémenter et intégrer des modèles de machine learning, notamment les réseaux de neurones convolutionnels (CNN), les réseaux de neurones à long terme (LSTM), et les classificateurs RandomForest, pour améliorer la précision de la reconnaissance des gestes et leur traduction.

Recherche et Analyse des Technologies Existantes : Étudier les bases de la langue des signes, identifier les gestes les plus courants et explorer les technologies actuelles de reconnaissance de gestes et d'intelligence artificielle qui peuvent être appliquées à la traduction de la langue des signes.

Validation et Amélioration : Effectuer des tests et validations pour garantir la performance et l'efficacité de l'application. Organiser des sessions de tests utilisateurs pour recueillir des retours et apporter les ajustements nécessaires pour améliorer l'application.

Documentation et Formation : Rédiger une documentation technique détaillée pour l'application et créer un guide utilisateur pour faciliter la prise en main par les personnes sourdes et malentendantes.

Ce stage permettra de développer des compétences en développement mobile, en intelligence artificielle, et en conception d'interfaces utilisateur, tout en contribuant à une solution innovante pour améliorer l'inclusion sociale des personnes sourdes et malentendantes.

1.3 Compétences acquises

Programmation et Développement Mobile :

- Compétence dans le développement d'applications mobiles multiplateformes, création d'interfaces utilisateur réactives et accessibles pour les systèmes iOS et Android.

Reconnaissance de Gestes et Intelligence Artificielle :

- Maîtrise des techniques de reconnaissance de gestes et de modélisation avec des réseaux neuronaux pour interpréter les séquences gestuelles dans un contexte spatial et temporel.
- Expertise dans les algorithmes de classification et d'amélioration de la précision pour la reconnaissance des gestes, en utilisant des modèles statistiques et de machine learning.
- Compétence en prétraitement et préparation de données pour améliorer la qualité des modèles de reconnaissance.

Conception d'Interfaces Utilisateur (UX/UI) :

- Capacité à concevoir des interfaces utilisateur intuitives, conviviales et accessibles, tout en prenant en compte les besoins spécifiques des utilisateurs finaux.

Recherche et Analyse :

- Expertise dans l'étude des gestes et des langues des signes pour améliorer l'interprétation et la traduction des gestes.
- Analyse approfondie des solutions de reconnaissance gestuelle existantes et adaptation de ces solutions pour répondre aux besoins spécifiques.

Tests et Validation :

- Connaissance des méthodes de tests unitaires et d'intégration pour garantir le bon fonctionnement de l'application.
- Capacité à réaliser des tests utilisateurs pour recueillir des retours concrets et améliorer les fonctionnalités de l'application.

Documentation et Formation :

- Expérience dans la rédaction de documents techniques et de guides utilisateurs clairs et précis pour faciliter la prise en main de l'application.

1.4 Présentation globale de 4D Business Consulting

1.4.1 Introduction

4DBC est un cabinet de conseil global de confiance, créé en 2008 pour servir les régions du GCC, du Moyen-Orient et de l'Afrique du Nord. Spécialisé dans la gestion des services technologiques et l'externalisation, 4DBC s'appuie sur une équipe de consultants experts pour accompagner ses clients. Avec une expérience incomparable et des capacités étendues dans tous les secteurs et fonctions commerciales, 4DBC collabore étroitement avec ses clients pour les aider à atteindre des performances optimales, que ce soit dans le secteur privé ou public.

1.4.2 Nos Services

1.4.2.1 Gestion des Services Technologiques

4DBC propose une gestion intégrée des services technologiques, offrant des solutions adaptées aux besoins spécifiques des entreprises, qu'il s'agisse d'optimisation des infrastructures ou de transformation numérique.

1.4.2.2 Externalisation

Nous fournissons des services d'externalisation qui permettent à nos clients de se concentrer sur leur cœur de métier tout en bénéficiant d'un soutien stratégique et opérationnel de haute qualité.

1.4.2.3 Stratégies Personnalisées

Fondé sur le principe que les résultats doivent primer sur les simples rapports, 4DBC se distingue en développant des stratégies personnalisées qui permettent à ses clients de surpasser leurs concurrents.

1.4.3 Notre Approche

1.4.3.1 Collaboration avec les Clients

4DBC croit fermement à une collaboration étroite avec ses clients, travaillant main dans la main pour comprendre leurs besoins uniques et élaborer des solutions sur mesure.

1.4.3.2 Résultats et Performances

Notre objectif est d'aider nos clients à atteindre des niveaux de performance élevés, en mettant l'accent sur des résultats tangibles et mesurables.

1.4.3.3 Innovation et Développement Stratégique

Nous innovons constamment pour répondre aux défis actuels du marché, en développant des stratégies qui positionnent nos clients en leaders de leur secteur.

Chapitre 2

Étude et Recherche

2.1 Base de Données de la Langue des Signes

Dans le cadre de ce projet, nous avons développé une base de données complète pour la reconnaissance de la langue des signes. Cette base de données est constituée de vidéos capturant différents gestes, qui sont ensuite converties en séquences de points de repère pour faciliter l'apprentissage automatique.

2.1.1 Structure de la Base de Données

La base de données contient 30 gestes différents, chacun enregistré à partir de vidéos capturées par une caméra. Chaque geste est représenté par 30 séquences, et chaque séquence correspond à un ensemble de points de repère extraits à l'aide de MediaPipe Holistic, un outil permettant de détecter les mouvements du corps entier, y compris les mains et le visage.

Les étapes clés de la construction de la base de données sont les suivantes :

- **Capture des Gestes** : Les gestes sont capturés à l'aide d'une caméra, puis traités par le modèle MediaPipe pour détecter les points de repère correspondant aux mouvements du corps, du visage, et des mains.
- **Extraction des Points de Repère** : Pour chaque geste, les points de repère (coordonnées x , y , z et visibilité) des poses, du visage, et des deux mains sont extraits. Si certains points de repère ne sont pas détectés, des vecteurs de zéros sont utilisés en remplacement pour garantir l'uniformité des données.
- **Sauvegarde des Données** : Chaque geste est sauvegardé sous forme de tableau `numpy`,

facilitant ainsi le traitement ultérieur des données pour l'entraînement des modèles de machine learning.

2.1.2 Exemple de Code pour la Capture des Données

Le code suivant illustre le processus de capture et d'extraction des points de repère à partir des vidéos des gestes. Le modèle MediaPipe est utilisé pour détecter les différentes parties du corps, et les points de repère sont ensuite extraits et stockés sous forme de tableau `numpy`.

```
import cv2          # OpenCV : utilisé pour la capture vidéo et le traitement d'images, notamment pour lire des vidéos/images et traiter des flux vidéo.
import numpy as np  # Numpy : utilisé pour le calcul scientifique et la manipulation de tableaux multidimensionnels, très utile dans le traitement d'images.
import os          # OS : permet d'interagir avec le système de fichiers, par exemple pour naviguer dans les répertoires ou lire des fichiers.
from matplotlib import pyplot as plt # Matplotlib : utilisé pour la visualisation des données, par exemple pour afficher des images ou des graphiques.
import time        # Time : permet de gérer les opérations liées au temps, comme créer des pauses ou mesurer la durée de certaines opérations.
import mediapipe as mp # MediaPipe : utilisé pour la détection de corps, des gestes, des mains et d'autres fonctionnalités liées à la reconnaissance vidéo.
from sklearn.model_selection import train_test_split # Scikit-learn : utilisé pour diviser les données en ensembles d'entraînement et de test.
from tensorflow.keras.utils import to_categorical # TensorFlow/Keras : utilisé pour convertir les étiquettes en matrices catégoriques pour l'apprentissage automatique.
```

FIGURE 2.1 – Les bibliothèques utilisées

Chaque bibliothèque a un rôle spécifique pour gérer différents aspects de l'application, notamment la capture vidéo, le traitement d'image, la détection de gestes, et la gestion des modèles d'apprentissage automatique.

```
# Initialiser MediaPipe Holistic, qui est une solution pour la détection du corps entier
mp_holistic = mp.solutions.holistic

# Initialiser MediaPipe Drawing Utils, qui contient des utilitaires pour dessiner les annotations
mp_drawing = mp.solutions.drawing_utils
```

FIGURE 2.2 – Points clés utilisant MP Holistic

Le code ci-dessus initialise les solutions MediaPipe pour la détection du corps entier et le dessin des annotations. MediaPipe Holistic est une puissante solution qui permet de détecter des points clés sur tout le corps, y compris les mains, le visage et la posture, à partir de vidéos ou d'images. MediaPipe Drawing Utils, quant à lui, fournit des outils pratiques pour visualiser ces points clés en dessinant des annotations sur les images. Ces deux modules, combinés, permettent de capturer et de représenter visuellement les mouvements et gestes d'un individu, ce qui est essentiel pour des applications comme la traduction de la langue des signes, l'analyse de mouvement, ou d'autres interactions basées sur les gestes.

```
def draw_styled_landmarks(image, results):
    # Dessiner les connexions du visage
    mp_drawing.draw_landmarks(image, results.face_landmarks, mp_holistic.FACEMESH_TESSELATION,
                              # Spécifications pour colorer les points de repère (points) du visage
                              mp_drawing.DrawingSpec(color=(80,110,10), thickness=1, circle_radius=1),
                              # Spécifications pour colorer les connexions (lignes) du visage
                              mp_drawing.DrawingSpec(color=(80,256,121), thickness=1, circle_radius=1)
                              )

    # Dessiner les connexions de la pose (corps)
    mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,
                              # Spécifications pour colorer les points de repère (points) du corps
                              mp_drawing.DrawingSpec(color=(80,22,10), thickness=2, circle_radius=4),
                              # Spécifications pour colorer les connexions (lignes) du corps
                              mp_drawing.DrawingSpec(color=(80,44,121), thickness=2, circle_radius=2)
                              )

    # Dessiner les connexions de la main gauche
    mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              # Spécifications pour colorer les points de repère (points) de la main gauche
                              mp_drawing.DrawingSpec(color=(121,22,76), thickness=2, circle_radius=4),
                              # Spécifications pour colorer les connexions (lignes) de la main gauche
                              mp_drawing.DrawingSpec(color=(121,44,250), thickness=2, circle_radius=2)
                              )

    # Dessiner les connexions de la main droite
    mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              # Spécifications pour colorer les points de repère (points) de la main droite
                              mp_drawing.DrawingSpec(color=(245,117,66), thickness=2, circle_radius=4),
                              # Spécifications pour colorer les connexions (lignes) de la main droite
                              mp_drawing.DrawingSpec(color=(245,66,230), thickness=2, circle_radius=2)
                              )
```

FIGURE 2.3 – fonction draw styled landmarks

```
def mediapipe_detection(image, model):
    # Convertir l'image de BGR en RGB
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # COLOR CONVERSION BGR 2 RGB

    # Rendre l'image non modifiable (non writeable)
    image.flags.writeable = False # Image is no longer writeable

    # Faire la prédiction en utilisant le modèle MediaPipe
    results = model.process(image) # Make prediction

    # Rendre l'image modifiable à nouveau
    image.flags.writeable = True # Image is now writeable

    # Convertir l'image de RGB en BGR
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR) # COLOR CONVERSION RGB 2 BGR

    # Retourner l'image et les résultats de la prédiction
    return image, results
```

FIGURE 2.4 – fonction mediapipe detection

```
def draw_landmarks(image, results):
    # Dessiner les connexions du visage
    mp_drawing.draw_landmarks(image, results.face_landmarks, mp_holistic.FACEMESH_TESSELATION) # Draw face connections

    # Dessiner les connexions de la pose (corps)
    mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS) # Draw pose connections

    # Dessiner les connexions de la main gauche
    mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS) # Draw left hand connections

    # Dessiner les connexions de la main droite
    mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS) # Draw right hand connections

    # draw_landmarks -> Function does not return the image but rather applies the landmark visualizations to the current image in place
```

FIGURE 2.5 – fonction draw landmarks

Ces trois fonctions, développées dans le cadre de l'utilisation de la bibliothèque MediaPipe et OpenCV, jouent un rôle essentiel dans la détection des corps et la visualisation des points de repère sur une image.

La fonction `mediapipe_detection` permet de préparer une image en convertissant son espace colorimétrique, d'effectuer des prédictions via un modèle MediaPipe, puis de restaurer l'image pour de nouvelles modifications, tout en renvoyant les résultats de la prédiction. Cette étape est cruciale pour le traitement des images en temps réel.

La fonction `draw_landmarks` se charge de dessiner les connexions et les points de repère (*landmarks*) détectés pour le visage, le corps et les mains sur une image. Ce processus ajoute une représentation visuelle des points clés du corps humain.

Enfin, la fonction `draw_styled_landmarks` personnalise l'apparence de ces points de repère en ajoutant des styles spécifiques tels que des couleurs et des épaisseurs différentes pour chaque partie du corps. Cela améliore l'affichage des résultats pour des visualisations plus claires et esthétiques.

Ces fonctions combinées permettent de transformer les résultats bruts du modèle de détection en une représentation visuelle claire et stylisée, facilitant ainsi la compréhension et l'interprétation des mouvements corporels pour des applications de détection de gestes, de suivi du mouvement ou d'analyse biométrique. En offrant une visualisation détaillée et personnalisée, elles améliorent la précision des analyses et la réactivité des systèmes basés sur la détection de mouvements.

```
# Initialiser la capture vidéo à partir de la webcam
cap = cv2.VideoCapture(0)

# Définir le modèle MediaPipe Holistic avec des seuils de détection et de suivi
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:
    while cap.isOpened():
        # Lire le flux vidéo
        ret, frame = cap.read()

        if not ret:
            continue

        # Faire les détectations
        image, results = mediapipe_detection(frame, holistic)
        print(results)

        # Dessiner les points de repère
        draw_styled_landmarks(image, results) # Utiliser la fonction améliorée pour dessiner les points de repère avec des couleurs différentes

        # Afficher à l'écran
        cv2.imshow('OpenCV Feed', image)

        # Quitter proprement
        if cv2.waitKey(10) & 0xFF == ord('q'):
            break

# Libérer la capture vidéo et fermer les fenêtres
cap.release()
cv2.destroyAllWindows()
# Compter le nombre de points de repère détectés sur la pose (corps)
num_pose_landmarks = len(results.pose_landmarks.landmark)
results
# Appeler la fonction pour dessiner les points de repère sur le visage, le corps, et les mains
draw_landmarks(frame, results)
```

FIGURE 2.6 – capture un flux vidéo en temps réel

Ce code capture un flux vidéo en temps réel à partir de la webcam, utilise *MediaPipe Holistic* pour détecter les points de repère du visage, du corps et des mains, et les affiche en superposition sur l'image. Les résultats sont visualisés avec des styles personnalisés pour faciliter

l'interprétation des mouvements. L'affichage continue jusqu'à ce que l'utilisateur appuie sur la touche 'q' pour quitter.

```
pose = []
# Pour chaque point de repère de la pose détecté, crée un tableau numpy avec les coordonnées x, y, z et la visibilité, puis ajoute-le à la liste 'pose'.
for res in results.pose_landmarks.landmark:
    test = np.array([res.x, res.y, res.z, res.visibility])
    pose.append(test)

# Crée un tableau numpy plat avec les coordonnées x, y, z et la visibilité des points de repère de la pose; sinon, initialise un tableau de zéros de taille 132 si aucun point de repère n'est détecté.
pose = np.array([[res.x, res.y, res.z, res.visibility] for res in results.pose_landmarks.landmark]).flatten() if results.pose_landmarks else np.zeros(132)
# Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère du visage; sinon, initialise un tableau de zéros de taille 1404 si aucun point de repère n'est détecté.
face = np.array([[res.x, res.y, res.z] for res in results.face_landmarks.landmark]).flatten() if results.face_landmarks else np.zeros(1404)
# Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère de la main gauche; sinon, initialise un tableau de zéros de taille 63 si aucun point de repère n'est détecté.
lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten() if results.left_hand_landmarks else np.zeros(21*3)
# Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère de la main droite; sinon, initialise un tableau de zéros de taille 63 si aucun point de repère n'est détecté.
rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten() if results.right_hand_landmarks else np.zeros(21*3)

def extract_keypoints(results):
    # Crée un tableau numpy plat avec les coordonnées x, y, z et la visibilité des points de repère de la pose; sinon, initialise un tableau de zéros de taille 132 si aucun point de repère n'est détecté.
    pose = np.array([[res.x, res.y, res.z, res.visibility] for res in results.pose_landmarks.landmark]).flatten() if results.pose_landmarks else np.zeros(33*4)

    # Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère du visage; sinon, initialise un tableau de zéros de taille 1404 si aucun point de repère n'est détecté.
    face = np.array([[res.x, res.y, res.z] for res in results.face_landmarks.landmark]).flatten() if results.face_landmarks else np.zeros(468*3)

    # Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère de la main gauche; sinon, initialise un tableau de zéros de taille 63 si aucun point de repère n'est détecté.
    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten() if results.left_hand_landmarks else np.zeros(21*3)

    # Crée un tableau numpy plat avec les coordonnées x, y, z des points de repère de la main droite; sinon, initialise un tableau de zéros de taille 63 si aucun point de repère n'est détecté.
    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten() if results.right_hand_landmarks else np.zeros(21*3)

    # Concatène tous les tableaux de points de repère (pose, visage, main gauche, main droite) en un seul tableau numpy.
    return np.concatenate([pose, face, lh, rh])

result_test = extract_keypoints(results)
```

FIGURE 2.7 – Extraction et Normalisation des Points de Repère

Ce code ci-dessus extrait les coordonnées et les caractéristiques des points de repère détectés par MediaPipe pour les poses, le visage, et les mains, et les organise dans des tableaux NumPy. Chaque tableau est initialisé avec des zéros si aucun point de repère n'est détecté. Les tableaux de points de repère sont ensuite concaténés pour créer une représentation complète et normalisée des données de détection, facilitant ainsi l'analyse ultérieure.

```
# en enregistrant chaque image sous forme de tableau numpy à l'intérieur de notre dossier 'MP_Data', nous aurons donc 30 tableaux numpy dans chaque dossier d'actions
np.save('0', result_test)
np.load('0.npy')

# Path for exported data, numpy arrays
DATA_PATH = os.path.join('MP_Data')

# Actions that we try to detect, including Tunisian Sign Language actions in French (transliterated)
actions = np.array(['n3ass','lunettes','je froid','salam', 'Chokran', 'je t aime', 'La', 'j aime', 'j aime pas', 'Nakel', 'Nchrob', 'parfait',
    'Telephone', 's il te plait', 'pleur','fort', 'lissar','Wagtach',
    'limine', 'sghir', 'kbir', 'stop', 'sma3ni','fakar','chof'])

#Trente vidéos de données
no_sequences = 30

# Les vidéos dureront 30 images
sequence_length = 30

# Vérifie si le chemin de répertoire spécifié par DATA_PATH n'existe pas.
# Si le répertoire n'existe pas, il le crée.
if not os.path.exists(DATA_PATH):
    os.makedirs(DATA_PATH)

# Itère sur chaque action dans la liste d'actions
for action in actions:
    # Pour chaque action, itère sur le nombre de séquences
    for sequence in range(no_sequences):
        try:
            # Essaie de créer un répertoire pour l'action et la séquence spécifiée dans DATA_PATH
            os.makedirs(os.path.join(DATA_PATH, action, str(sequence)))
        except:
            # Ignore les erreurs (par exemple, si le répertoire existe déjà) et continue
            pass
```

FIGURE 2.8 – enregistre les images sous forme de tableaux NumPy

Ce code enregistre les images sous forme de tableaux NumPy dans des dossiers spécifiques pour chaque action, en utilisant le chemin 'MPData'. Les actions détectées incluent des signes en langue des signes. Il crée des répertoires pour chaque action et chaque séquence, avec un total de 30 séquences par action, et vérifie la présence des répertoires avant de les créer pour stocker les données vidéo.

```
# Ouvre une connexion à la caméra (0 pour la caméra par défaut)
cap = cv2.VideoCapture(0)
# Initialise le modèle MediaPipe avec les seuils de confiance pour la détection et le suivi
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:

    # Boucle à travers chaque action définie
    for action in actions:
        # Boucle à travers chaque séquence (vidéo) pour l'action actuelle
        for sequence in range(no_sequences):
            # Boucle à travers chaque frame de la séquence (longueur de la séquence)
            for frame_num in range(sequence_length):

                # Lit un frame de la caméra
                ret, frame = cap.read()

                # Effectue la détection des points de repère avec le modèle MediaPipe
                image, results = mediapipe_detection(frame, holistic)
                # print(results) # (Commenté) Affiche les résultats de la détection
                # Dessine les points de repère stylisés sur l'image
                draw_styled_landmarks(image, results)
                # Applique une logique d'attente pour afficher un message
                if frame_num == 0:
                    # Affiche un message indiquant le début de la collecte de données
                    cv2.putText(image, 'STARTING COLLECTION', (120,200), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255, 0), 4, cv2.LINE_AA)
                    # Affiche des informations supplémentaires sur la collecte des frames
                    cv2.putText(image, 'Collecting frames for {} Video Number {}'.format(action, sequence), (15,12), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1, cv2.LINE_AA)
                    # Affiche l'image avec les messages à l'écran
                    cv2.imshow('OpenCV Feed', image)
                    # Attend 2 secondes avant de passer à la frame suivante
                    cv2.waitKey(2000)
                else:
                    # Affiche uniquement les informations sur la collecte des frames
                    cv2.putText(image, 'Collecting frames for {} Video Number {}'.format(action, sequence), (15,12), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1, cv2.LINE_AA)
                    # Affiche l'image avec les messages à l'écran
                    cv2.imshow('OpenCV Feed', image)

                # Extrait les points de repère clés à partir des résultats de la détection
                keypoints = extract_keypoints(results)
                # Crée un chemin de fichier pour sauvegarder les points de repère extraits
                npy_path = os.path.join(DATA_PATH, action, str(sequence), str(frame_num))
                # Sauvegarde les points de repère dans un fichier .npy
                np.save(npy_path, keypoints)

                # Permet de quitter la boucle en appuyant sur la touche 'q'
                if cv2.waitKey(10) & 0xFF == ord('q'):
                    break

# Libère la caméra et ferme toutes les fenêtres d'affichage OpenCV
cap.release()
cv2.destroyAllWindows()
```

FIGURE 2.9 – collecter les données

Le code ouvre une connexion à la caméra et initialise le modèle MediaPipe pour la détection des points de repère du corps. Il parcourt chaque action et séquence définie, capturant des vidéos à partir de la caméra. Pour chaque frame, il effectue la détection des points de repère, les dessine sur l'image, et affiche des messages pour indiquer le début de la collecte des données. Les points de repère extraits sont sauvegardés sous forme de fichiers NumPy (.npy) dans des répertoires spécifiés pour chaque action et séquence. La collecte se poursuit jusqu'à ce que l'utilisateur appuie sur 'q', après quoi la caméra est libérée et toutes les fenêtres OpenCV sont fermées.

Voici un exemple de trois gestes :

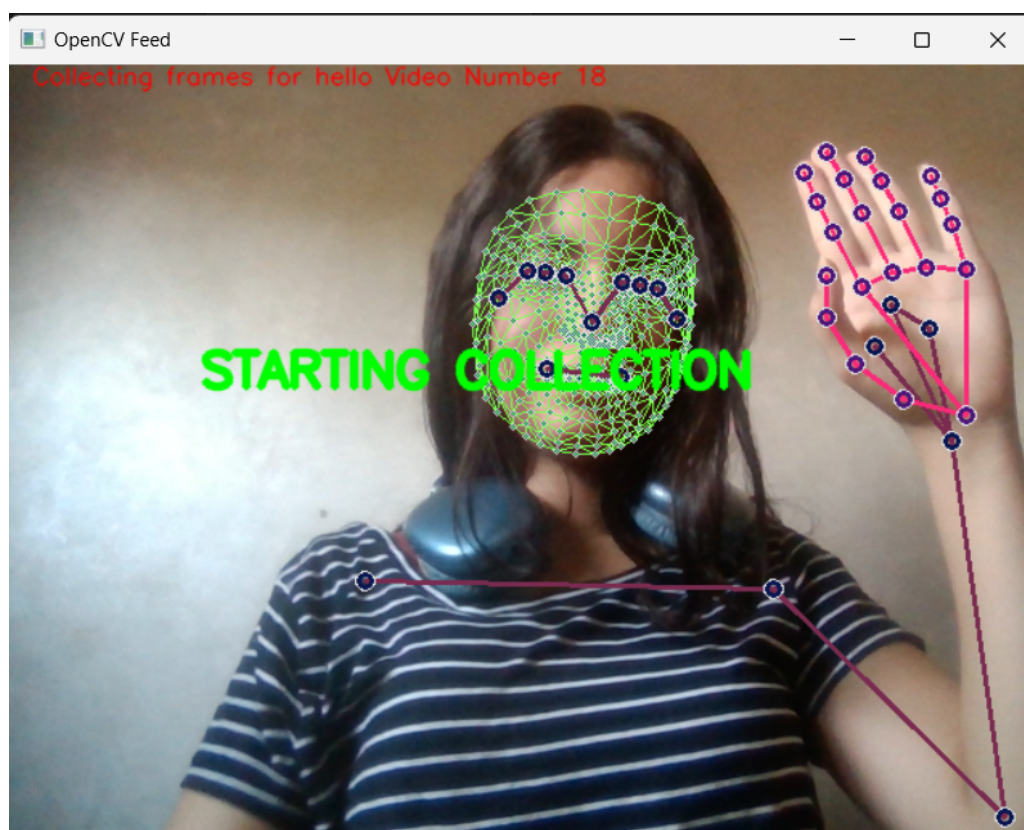


FIGURE 2.10 – geste 1

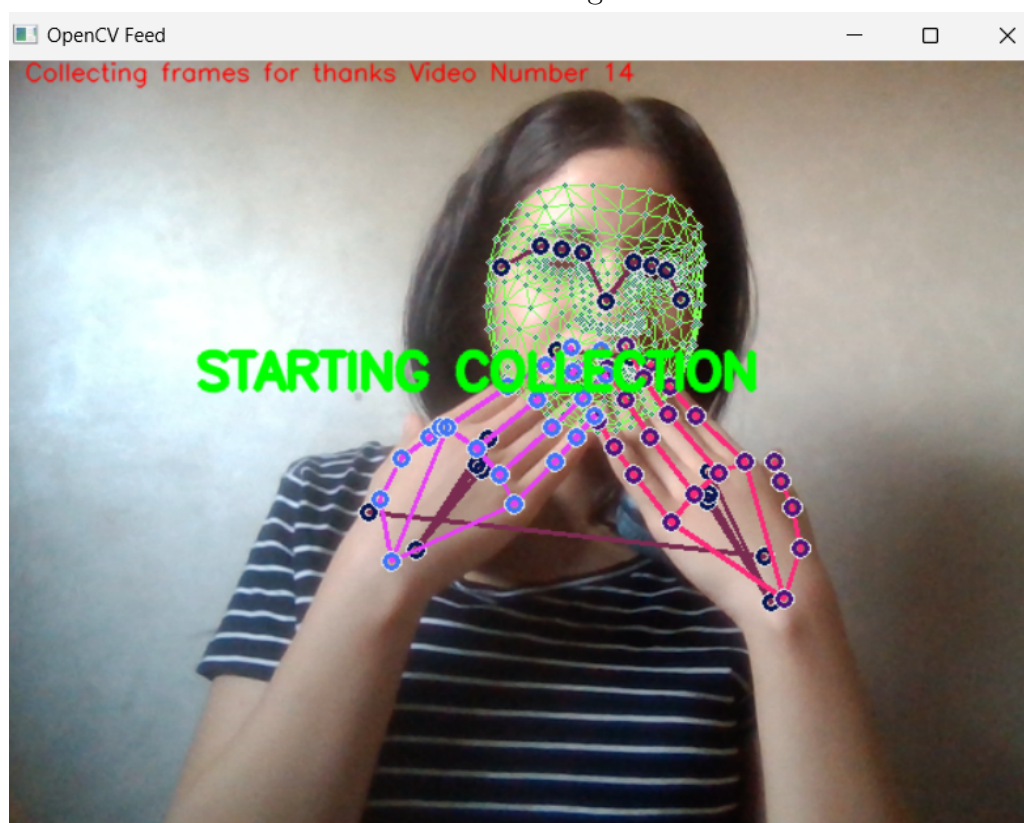


FIGURE 2.11 – geste 2

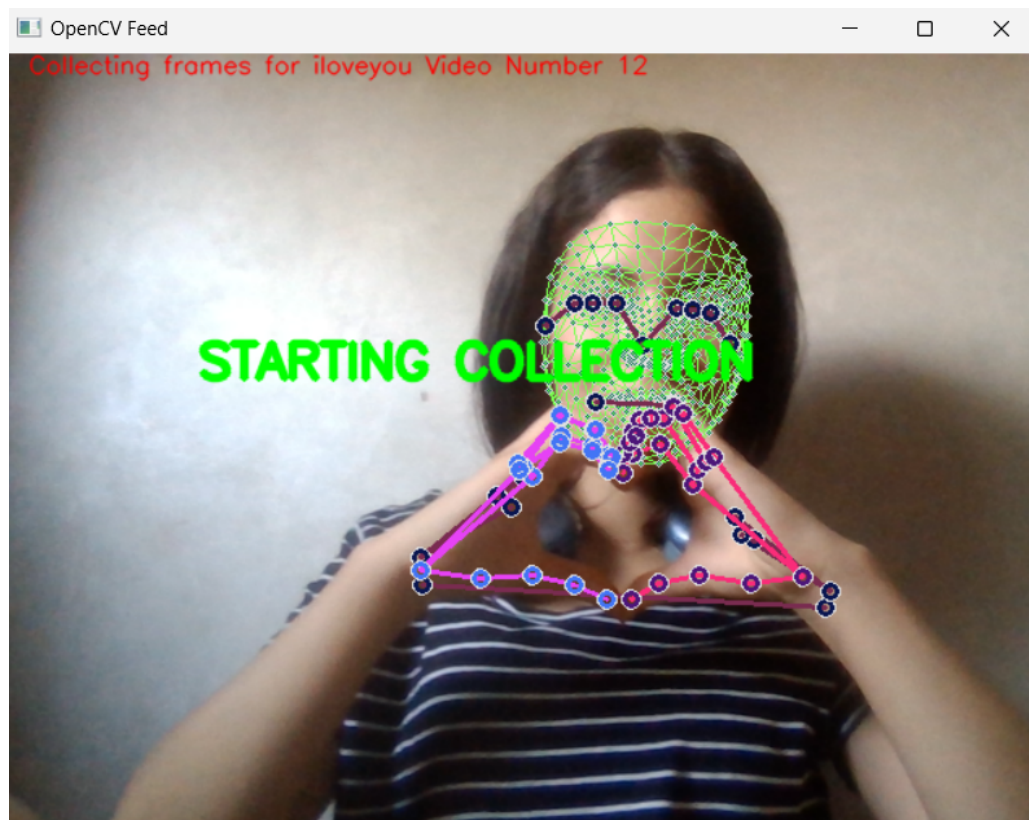


FIGURE 2.12 – geste 3

Pour chaque geste parmi les 30 gestes capturés, l'idée est de prendre 30 captures individuelles (ou frames) et de les organiser pour construire 30 séquences distinctes. L'objectif est de créer un dataset riche et varié pour chaque geste en capturant des séquences d'images représentant le geste à différentes étapes de son exécution. Ces séquences permettront à un modèle d'intelligence artificielle (IA) d'apprendre à reconnaître le geste, non seulement sur la base d'une image unique, mais aussi en tenant compte de la continuité et de la temporalité des mouvements.

Le processus détaillé se déroule comme suit :

1. Capturer chaque geste :

- Pour chaque geste, une vidéo est enregistrée ou des images sont capturées à intervalle régulier pendant que le geste est exécuté.
- Chaque vidéo ou geste est divisé en **30 captures (frames)**, représentant les différentes étapes du geste. Ces captures peuvent être prises à intervalles réguliers pour couvrir le début, le milieu et la fin de l'exécution du geste.

2. Construire des séquences :

- Chaque série de 30 images devient une **séquence** de gestes. Une séquence correspond à une série ordonnée de frames qui représente le mouvement complet du geste.
- Par exemple, pour le geste *bonjour*, 30 captures peuvent illustrer le mouvement de la main depuis le début de la salutation jusqu'à la fin.

3. Répliquer pour 30 gestes :

- Le processus est répété pour **30 gestes distincts**, chacun étant capturé sous forme de **30 séquences**.
- Cela donne au total **900 séquences** ($30 \text{ gestes} \times 30 \text{ séquences par geste}$), qui pourront ensuite être utilisées pour entraîner un modèle de reconnaissance de gestes.

Les avantages de ce processus sont nombreux :

- **Amélioration de la précision** : En capturant plusieurs étapes du geste, le modèle peut mieux comprendre la dynamique du mouvement et améliorer la précision de la reconnaissance.
- **Prise en compte du contexte temporel** : Le modèle apprend non seulement à identifier les gestes à partir d'une seule image, mais également à comprendre le déroulement dans le temps, ce qui est essentiel pour la reconnaissance de gestes complexes.
- **Robustesse face aux variations** : En capturant plusieurs séquences pour chaque geste, le modèle peut mieux gérer les variations dans la manière dont un geste est exécuté par différentes personnes.

Ces séquences peuvent être utilisées dans un **modèle de reconnaissance gestuelle** basé sur des réseaux neuronaux récurrents (comme LSTM ou GRU) ou des architectures qui tiennent compte de la temporalité, tels que les **CNN 3D** ou les **Transformers**. Cela peut également être appliqué pour améliorer la traduction en temps réel des gestes en texte ou en paroles dans des applications destinées à la communication pour les personnes sourdes et malentendantes.

Ce processus permet donc de capturer toute la richesse du mouvement gestuel et de fournir au modèle d'apprentissage des données pertinentes pour une reconnaissance précise et robuste.

```
label_map = {label:num for num, label in enumerate(actions)}

# Initialise deux listes vides pour stocker les séquences et les étiquettes
sequences, labels = [], []

# Itère à travers chaque action dans la liste d'actions
for action in actions:
    # Pour chaque action, itère à travers chaque séquence (vidéo)
    for sequence in range(no_sequences):
        # Crée une liste pour stocker les frames de la séquence actuelle
        window = []
        # Itère à travers chaque frame de la séquence
        for frame_num in range(sequence_length):
            # Charge les points de repère extraits (au format .npy) pour la frame courante
            res = np.load(os.path.join(DATA_PATH, action, str(sequence), "{}.npy".format(frame_num)))
            # Ajoute les points de repère à la fenêtre (séquence)
            window.append(res)
        # Ajoute la fenêtre (séquence complète) à la liste des séquences
        sequences.append(window)
        # Ajoute l'étiquette correspondant à l'action à la liste des étiquettes
        labels.append(label_map[action])

X = np.array(sequences)
y = to_categorical(labels).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.05)
```

FIGURE 2.13 – Preparation des données

Ce code prépare les données pour l'entraînement d'un modèle de détection de gestes. Il commence par créer un mappage des étiquettes d'action en numéros pour une gestion facile des

classes. Ensuite, il parcourt chaque action et chaque séquence pour charger les points de repère extraits, les organisant en fenêtres de séquences. Ces séquences sont stockées dans une liste avec leurs étiquettes correspondantes.

2.1.3 Actions Détectées

Les résultats obtenus après la collecte et le prétraitement des données sont les suivants :

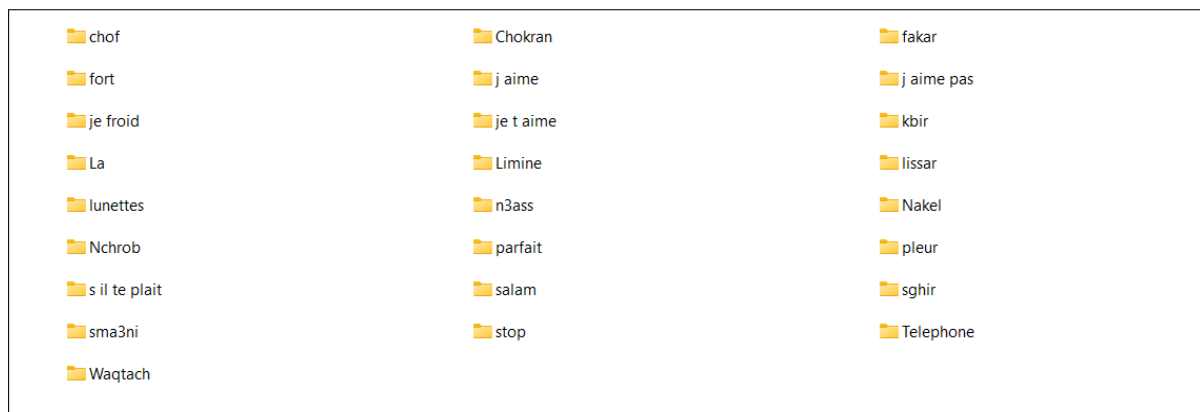


FIGURE 2.14 – Base de données

Pour chaque action détectée, 30 dossiers sont générés, et chaque dossier contient 30 fichiers au format .npy, représentant des séquences de mouvements capturés. Ces données servent à l'entraînement d'un modèle de reconnaissance de gestes, capable de convertir les mouvements en texte et en messages vocaux.

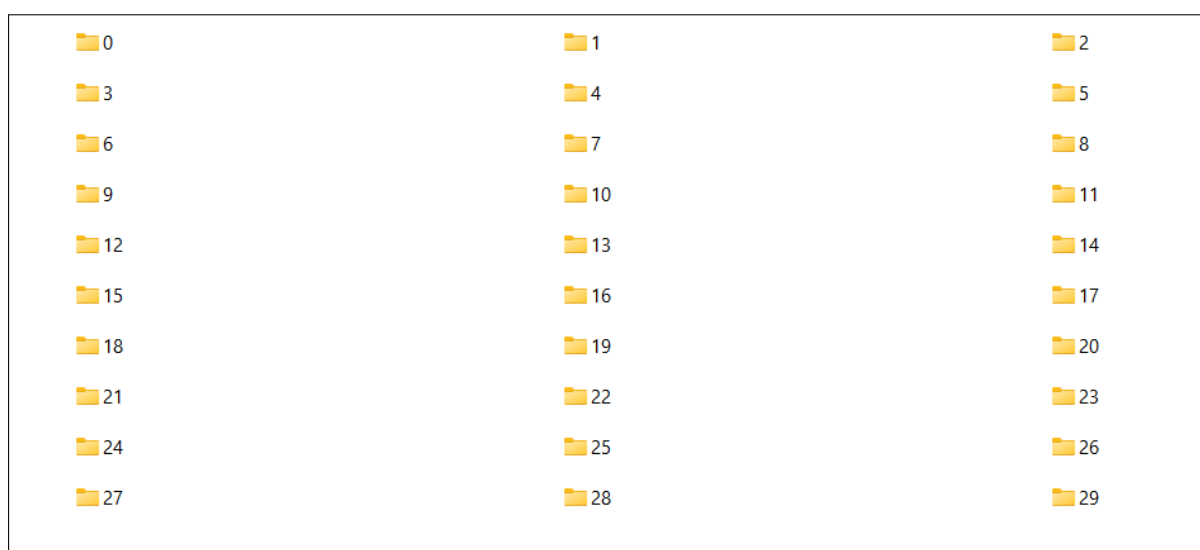


FIGURE 2.15 – Les 30 Séquences d'une action

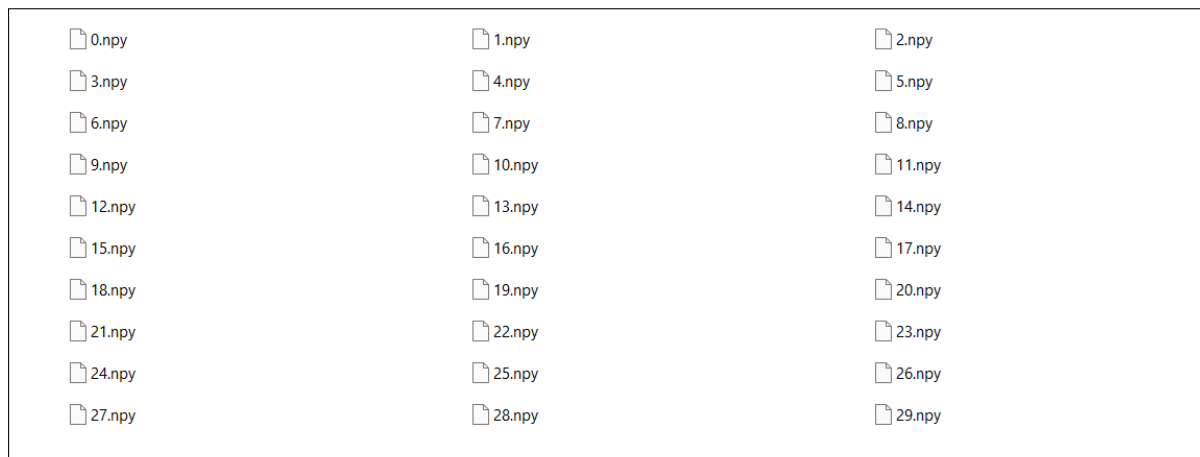


FIGURE 2.16 – Les 30 fichier de format .npy

Ces gestes constituent l'ensemble des données de base utilisées pour entraîner un modèle de reconnaissance, capable de traduire les mouvements détectés en texte et en messages vocaux.

2.2 Technologies Utilisées

Pour la réalisation de ce projet, plusieurs technologies et outils ont été employés pour assurer le bon fonctionnement de l'application Flutter destinée à la reconnaissance des gestes, ainsi que la communication avec un backend Flask. Voici un aperçu des principales technologies utilisées :

2.2.1 Vision par Ordinateur

La vision par ordinateur est une discipline de l'intelligence artificielle qui permet aux machines de comprendre et d'interpréter le monde visuel à partir d'images ou de vidéos. Dans ce projet, elle est utilisée pour reconnaître les gestes capturés par la caméra du dispositif. Parmi Les Technologies et Outils Utilisés de Vision par Ordinateur :

- **Modèles de Reconnaissance de Gestes** : Utilisation de modèles de machine learning, notamment les réseaux de neurones convolutifs (CNN), pour détecter et classifier les gestes dans les images. Les CNN sont performants pour extraire des caractéristiques visuelles à partir des images.
- **Réseaux de Neurones à Long Terme (LSTM)** : Intégration des LSTM pour modéliser les aspects temporels des gestes, notamment pour ceux ayant des dynamiques temporelles complexes. Les LSTM sont utilisés pour modéliser les séquences temporelles et capturer les dépendances à long terme.

- **Bibliothèques de Vision par Ordinateur** : Utilisation de bibliothèques comme OpenCV ou TensorFlow pour le traitement d'images et l'extraction de caractéristiques nécessaires à la reconnaissance des gestes.
- **Entraînement des Modèles** : Entraînement des modèles sur des ensembles de données d'images de gestes pour améliorer la précision de la reconnaissance.

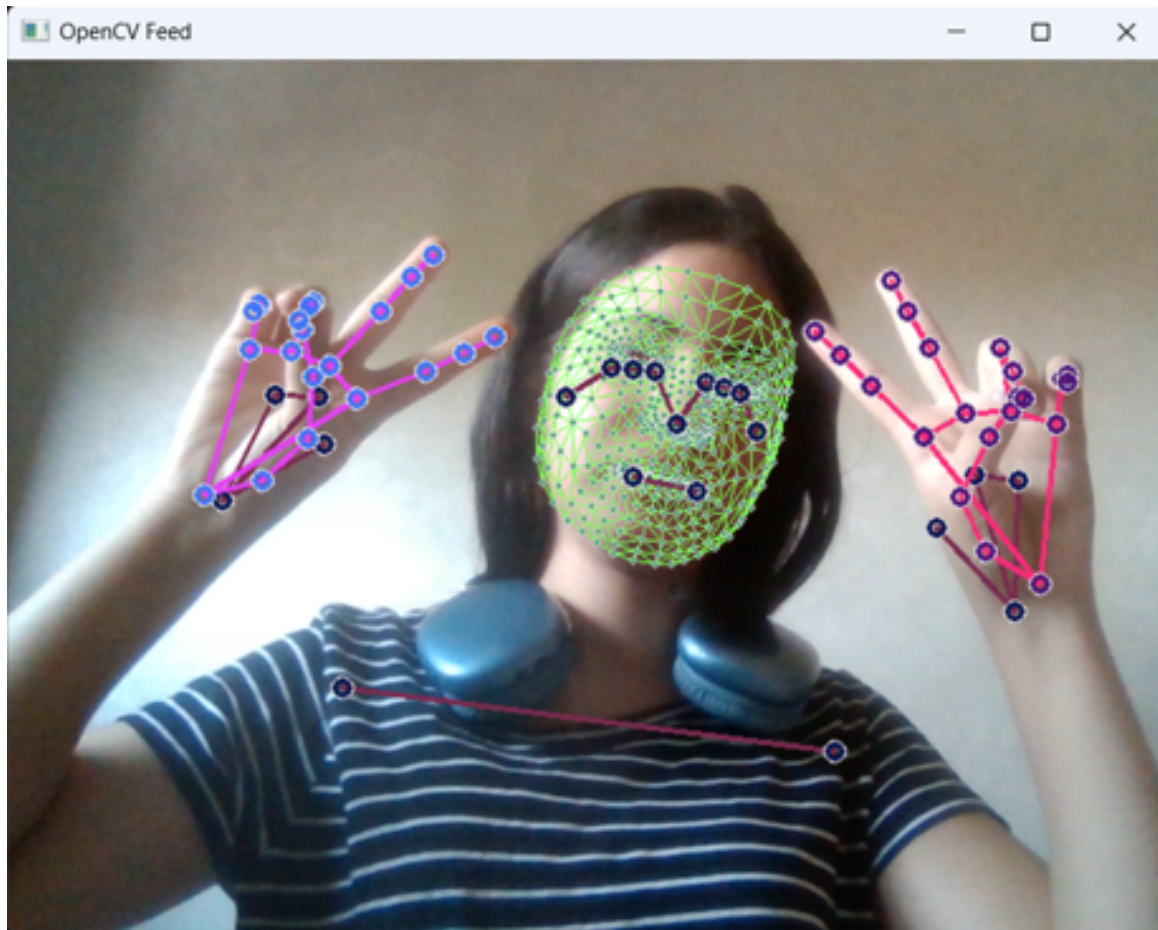


FIGURE 2.17 – Capture de geste

2.2.2 Flutter

Flutter est un framework open-source développé par Google, permettant de créer des applications multiplateformes à partir d'une seule base de code. Il utilise le langage Dart et offre une riche bibliothèque de widgets pour créer des interfaces utilisateur performantes. Parmi Les Fonctionnalités Utilisées de Flutter :

- **Widgets Flutter** : Utilisation des widgets comme `Scaffold`, `AppBar`, `TextField`, `IconButton`, et `CameraPreview` pour créer une interface utilisateur réactive et intuitive.
- **Gestion de la Caméra** : Intégration du package `camera` pour capturer des images à l'aide de la caméra du dispositif.

- **Traitement du Texte Vocal** : Utilisation du package `flutter_tts` pour convertir les prédictions en messages vocaux.

2.2.3 Dart

Dart est le langage de programmation utilisé par Flutter. Il est conçu pour être efficace dans le développement d'applications modernes, avec des performances optimisées et une syntaxe claire.

2.2.4 Flask

Flask est un micro-framework web écrit en Python. Il est utilisé pour créer l'API qui reçoit les images capturées, effectue la reconnaissance des gestes à l'aide d'un modèle pré-entraîné, et renvoie les résultats à l'application Flutter. Parmi Les Fonctionnalités Utilisées de Flask

- **API REST** : Création d'une API REST pour gérer les requêtes HTTP POST contenant les images des gestes.
- **Modèle de Reconnaissance des Gestes** : Utilisation d'un modèle de machine learning pour identifier les gestes à partir des images.
- **Gestion des Fichiers** : Traitement des fichiers envoyés par le client Flutter et gestion des réponses au format JSON.

2.2.5 Émulateur

L'outil d'émulation joue un rôle crucial dans le développement et les tests de l'application, permettant de simuler l'exécution sur différents appareils sans nécessiter de matériel physique.

- **Émulateur Android** : Utilisation de l'émulateur intégré à Android Studio pour tester l'application Flutter sur différents dispositifs et tailles d'écran. L'émulateur permet de simuler la capture de gestes via une caméra virtuelle et d'observer la réactivité de l'application dans diverses conditions.
- **Debugging et Tests** : L'émulateur fournit également un environnement sûr pour déboguer l'application en temps réel, tester la gestion des erreurs, et évaluer les performances du système sans avoir besoin de dispositifs physiques.

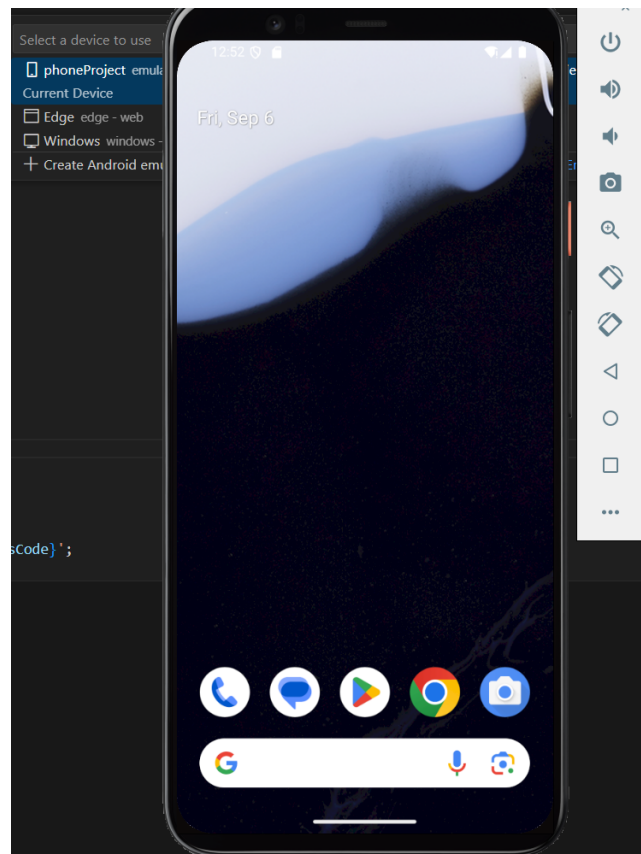


FIGURE 2.18 – Émulateur Android

2.2.6 Bibliothèques et Packages

- **http** : Package Dart utilisé pour effectuer des requêtes HTTP vers l'API Flask.
- **path_provider** : Package Dart pour accéder aux répertoires de fichiers temporaires sur le dispositif.

2.2.7 Environnement de Développement

- **IDE** : Utilisation d'Android Studio ou Visual Studio Code pour le développement de l'application Flutter.
- **Serveur Local** : Exécution de l'API Flask sur un serveur local pour faciliter les tests et le développement.
- **Émulateur** : Utilisation de l'émulateur Android pour simuler différentes configurations de dispositifs mobiles et tester l'application sans besoin de matériel physique.

Chapitre 3

Structuration et Modélisation

3.1 Diagramme de Cas d'Utilisation

Les diagrammes de Cas d'Utilisation sont des outils essentiels pour identifier les interactions entre les utilisateurs (acteurs) et le système. Il permet de visualiser les fonctions du système et la façon dont les différents acteurs interagissent avec elles. Ces schémas aident à identifier les besoins et les exigences du système en décrivant des cas d'utilisation spécifiques auxquels les acteurs peuvent accéder

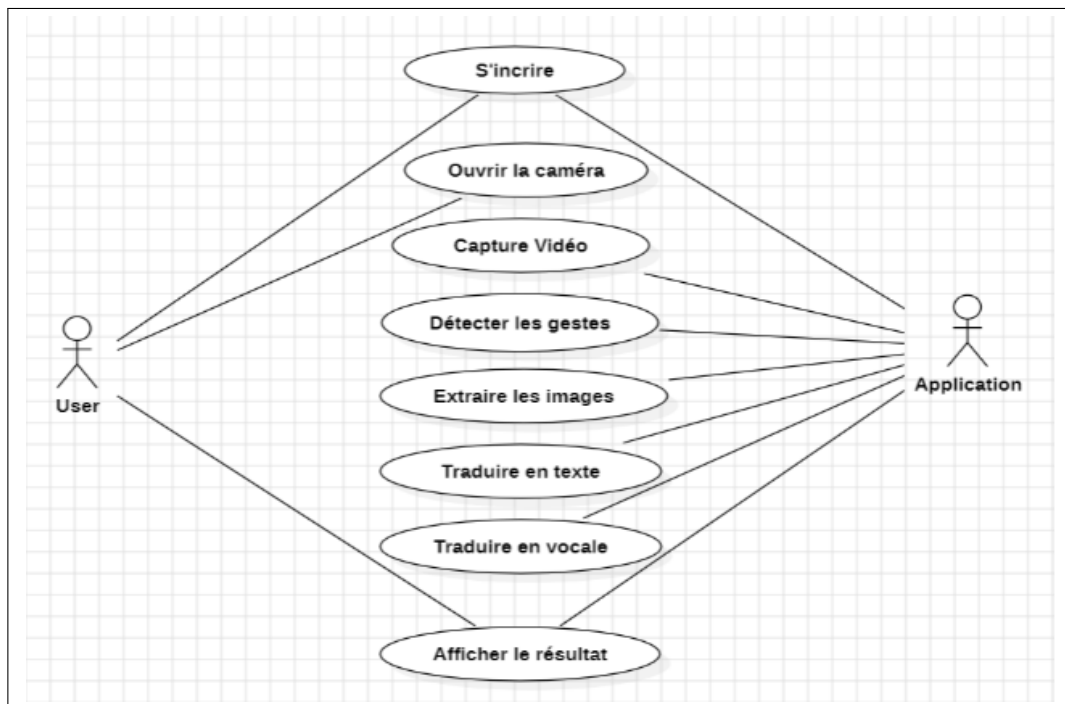


FIGURE 3.1 – Diagramme de Cas d'Utilisation

3.2 Diagramme de Séquence

Les diagrammes de séquence montrent comment les objets interagissent entre eux au fil du temps. Il clarifie l'ordre des messages échangés entre les objets pour accomplir une fonction particulière. Ces schémas sont utiles pour comprendre les processus dynamiques du système et pour décrire le comportement du système au fil du temps.

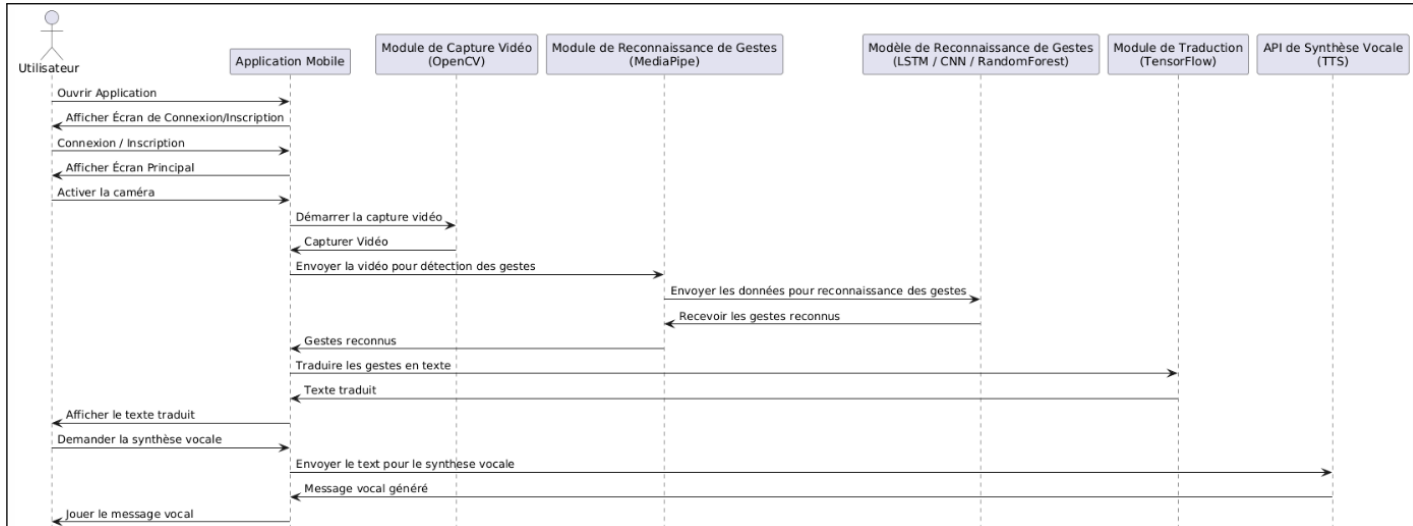


FIGURE 3.2 – Diagramme de Séquence

3.3 Diagramme de Classes

Les diagrammes de classe représentent les différentes catégories d'objets dans le système, ainsi que leurs attributs, méthodes et relations. Il fournit une vue cohérente du modèle de données du système et montre comment les différentes catégories interagissent et se connectent. Ces diagrammes aident à concevoir la structure du système et à réguler les éléments de données.

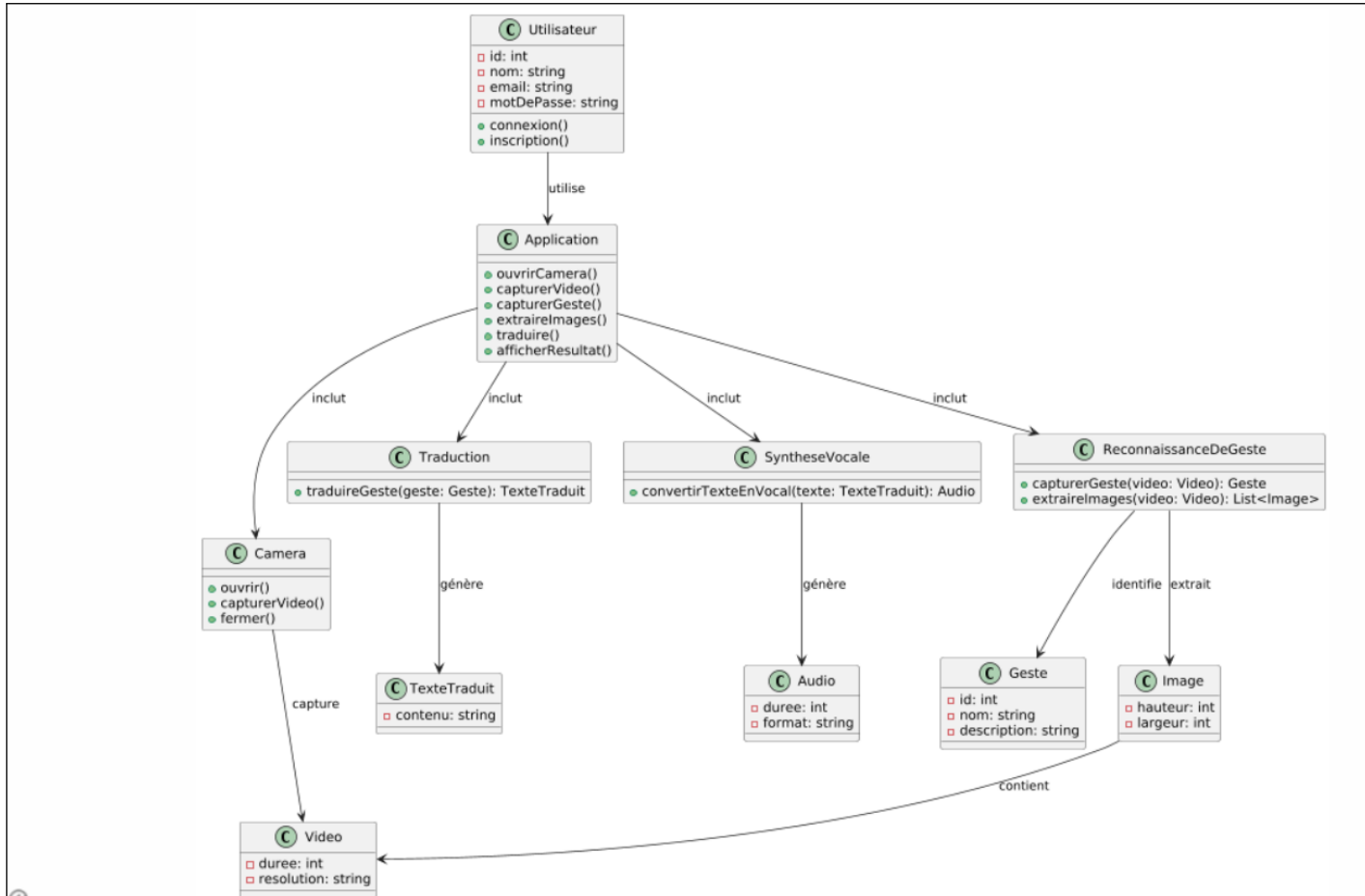


FIGURE 3.3 – Diagramme de Classes

3.4 Diagramme d'Activité

Les diagrammes d'activité représentent le fonctionnement et les opérations du système. Il décrit les différentes étapes d'un processus, les décisions et les mesures nécessaires pour accomplir une tâche spécifique. Ces diagrammes sont utiles pour la modélisation de processus et de flux de travail complexes au sein du système.

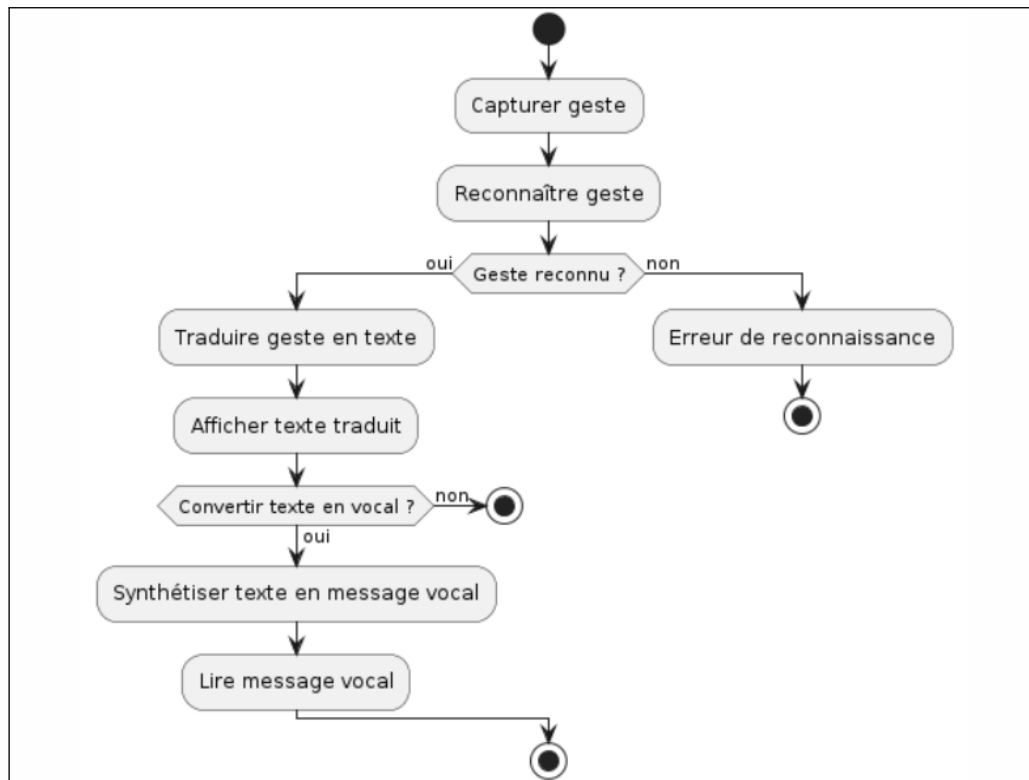


FIGURE 3.4 – Diagramme d'Activité

3.5 Diagramme d'État de Transition

Les diagrammes d'état de transition montrent les différentes conditions que le corps peut rencontrer au cours de son cycle de vie et les transitions entre ces conditions. Ils permettent de percevoir les changements dans la situation en réponse à des événements ou circonstances spécifiques. Ces schémas sont importants pour comprendre le comportement dynamique des choses et la logique de gestion des cas dans le système.

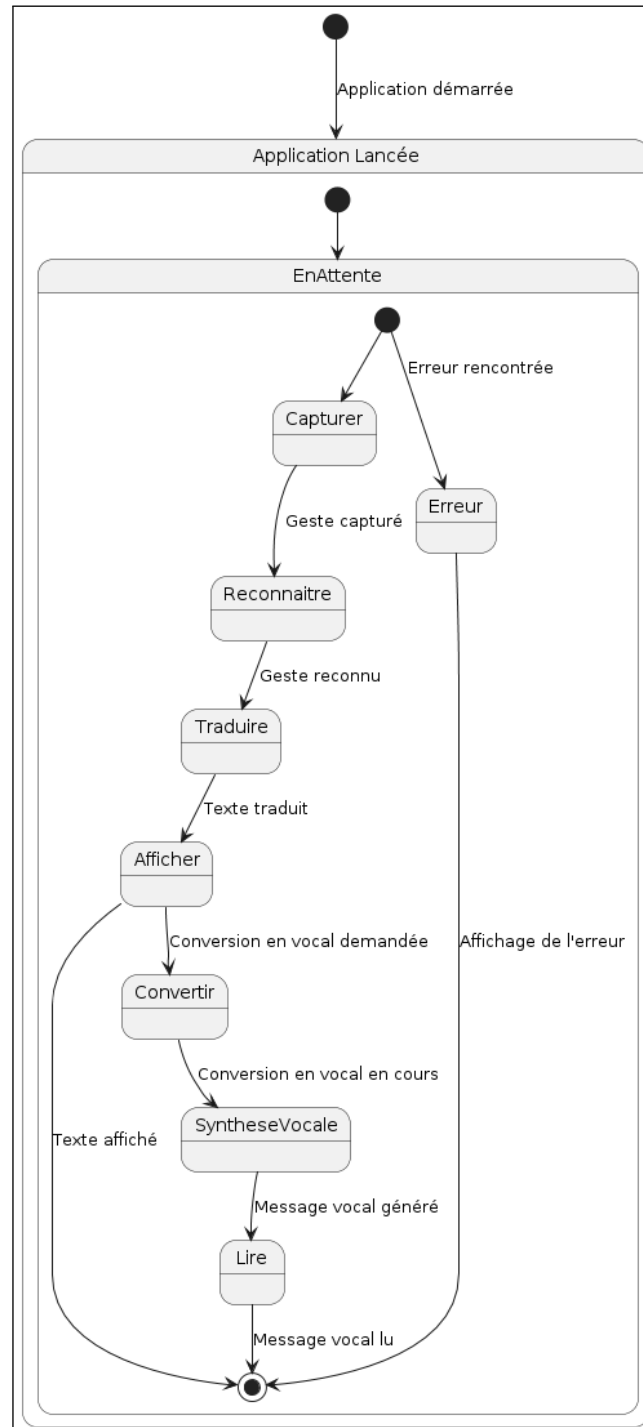


FIGURE 3.5 – Diagramme d'État de Transition

3.6 Diagramme de Flux des Données

Les diagrammes de flux de données (DFD) montrent le flux d'information dans le système, c'est-à-dire la façon dont les données sont entrées, traitées, stockées et sorties du système. Ces documents permettent de visualiser les processus de traitement des données et d'identifier les points d'entrée et de sortie de l'information.

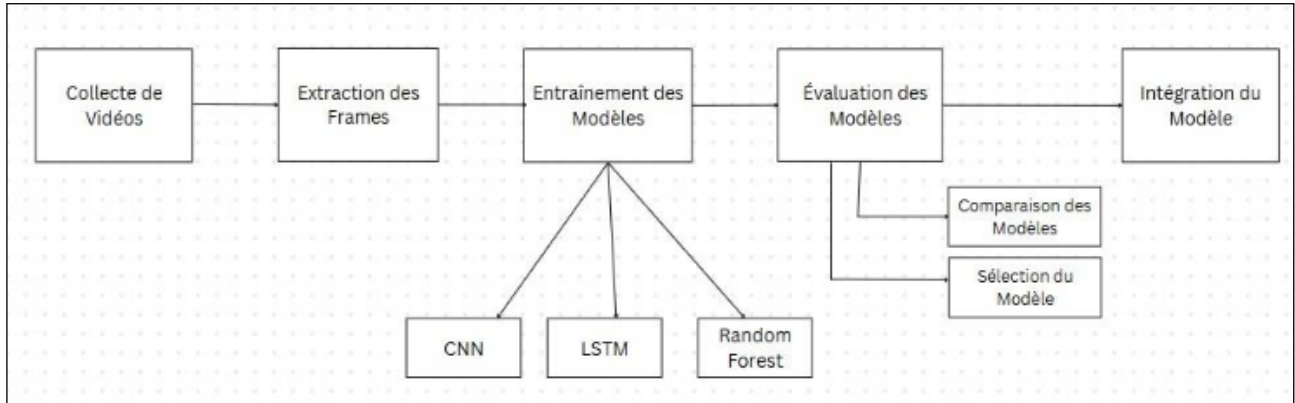


FIGURE 3.6 – Diagramme de Flux de Données

Chapitre 4

Partie Intelligence Artificielle

4.1 Développement et Test des Algorithmes

Dans ce projet, nous avons testé une variété de modèles de machine learning pour déterminer celui qui offre les meilleures performances en matière de reconnaissance des gestes. Plus précisément, nous avons évalué plus de huit modèles différents, incluant à la fois des modèles unitaires et des combinaisons de plusieurs modèles. L'objectif principal de cette démarche était de sélectionner le modèle qui garantit la meilleure fiabilité et performance pour l'application.

Pour évaluer la performance du modèle, plusieurs métriques sont employées pour obtenir une vue complète de sa précision et de son efficacité :

- **Accuracy** : La précision globale du modèle est calculée comme le pourcentage d'observations correctement classifiées par rapport au nombre total d'observations. La formule est :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

où TP est le nombre de vrais positifs, TN est le nombre de vrais négatifs, FP est le nombre de faux positifs, et FN est le nombre de faux négatifs.

- **Precision** : La précision évalue le rapport des exemples positifs correctement définis parmi toutes les prévisions positives fournies par le modèle. La formule est :

$$\text{Precision} = \frac{TP}{TP + FP}$$

où TP est le nombre de vrais positifs et FP est le nombre de faux positifs.

- **Recall (Sensitivity)** : Rappel ou sensibilité désigne la proportion d'exemples positifs

correctement définis sur tous les exemples positifs réels. La formule est :

$$\text{Recall} = \frac{TP}{TP + FN}$$

où TP est le nombre de vrais positifs et FN est le nombre de faux négatifs.

- **F1 Score** : Le score F1 est la moyenne harmonique de la précision et du rappel, offrant un équilibre entre les deux. La formule est :

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Confusion Matrix** : La matrice de confusion est un tableau résumant le rendement du modèle en indiquant le nombre de vrais positifs (TP), de faux positifs (FP), de vrais négatifs (TN) et de faux négatifs (FN). La structure de la matrice est généralement représentée comme suit :

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

Chaque élément de la matrice fournit des informations sur le rendement du modèle pour chaque catégorie.

Chaque modèle a été testé et comparé sur un ensemble de données représentatif afin de garantir que le modèle final choisi offre la meilleure précision et robustesse possible. Cette approche rigoureuse vise à assurer que l'application de reconnaissance des gestes fonctionne de manière fiable et efficace, en répondant aux besoins des utilisateurs de manière optimale.

4.1.1 Réseaux de Neurones Convolutifs (CNN 3D)

CNN 3D est une extension des réseaux traditionnels de CNN qui inclut une dimension supplémentaire d'analyse de séquençage d'image ou de tailles de données. Contrairement aux réseaux CNN bidimensionnels, qui traitent les images en deux dimensions, les réseaux CNN 3D utilisent des filtres de synthèse tridimensionnels pour capturer les caractéristiques spatiales et temporelles dans le séquençage des données. Cela le rend particulièrement efficace pour identifier les gestes 3D, permettant une analyse plus complète des mouvements et interactions dans l'espace 3D. En exploitant simultanément l'information spatiale et temporelle, les réseaux CNN 3D peuvent extraire des propriétés complexes des gestes et améliorer la précision et la durabilité des modèles d'identification.

```

79/79 2s 19ms/step - categorical_accuracy: 0.9502 - loss: 0.1295 - val_categorical_accuracy: 0.6970 - val_loss: 26.6370
Epoch 187/200
79/79 1s 18ms/step - categorical_accuracy: 0.9820 - loss: 0.0485 - val_categorical_accuracy: 0.5152 - val_loss: 26.9487
Epoch 188/200
79/79 2s 19ms/step - categorical_accuracy: 0.9881 - loss: 0.0510 - val_categorical_accuracy: 0.7576 - val_loss: 21.9881
Epoch 189/200
79/79 2s 18ms/step - categorical_accuracy: 0.9731 - loss: 0.1513 - val_categorical_accuracy: 0.3636 - val_loss: 32.6749
Epoch 190/200
79/79 1s 18ms/step - categorical_accuracy: 0.9754 - loss: 0.1013 - val_categorical_accuracy: 0.3333 - val_loss: 29.4961
Epoch 191/200
79/79 2s 19ms/step - categorical_accuracy: 0.9638 - loss: 0.1433 - val_categorical_accuracy: 0.7273 - val_loss: 18.6577
Epoch 192/200
79/79 2s 21ms/step - categorical_accuracy: 0.9833 - loss: 0.0568 - val_categorical_accuracy: 0.4848 - val_loss: 22.0614
Epoch 193/200
79/79 2s 20ms/step - categorical_accuracy: 0.9880 - loss: 0.0438 - val_categorical_accuracy: 0.5455 - val_loss: 20.9787
Epoch 194/200
79/79 2s 19ms/step - categorical_accuracy: 0.9899 - loss: 0.0368 - val_categorical_accuracy: 0.6364 - val_loss: 17.4185
Epoch 195/200
79/79 2s 19ms/step - categorical_accuracy: 0.9624 - loss: 0.1243 - val_categorical_accuracy: 0.4848 - val_loss: 24.5348
Epoch 196/200
79/79 1s 18ms/step - categorical_accuracy: 0.9461 - loss: 0.2084 - val_categorical_accuracy: 0.3939 - val_loss: 17.7591
Epoch 197/200
79/79 1s 17ms/step - categorical_accuracy: 0.9430 - loss: 0.1477 - val_categorical_accuracy: 0.5455 - val_loss: 16.3078
Epoch 198/200
79/79 1s 17ms/step - categorical_accuracy: 0.9492 - loss: 0.2233 - val_categorical_accuracy: 0.3636 - val_loss: 22.1093
Epoch 199/200
79/79 1s 17ms/step - categorical_accuracy: 0.9362 - loss: 0.1905 - val_categorical_accuracy: 0.5152 - val_loss: 17.9645
Epoch 200/200
79/79 2s 18ms/step - categorical_accuracy: 0.9467 - loss: 0.2231 - val_categorical_accuracy: 0.1212 - val_loss: 138.3613

```

FIGURE 4.1 – CNN-3D

Évaluer la Performance du Modèle :

Accuracy: 0.8181818181818182

Precision: 0.8888888888888888

Recall (Sensitivity): 0.8181818181818182

F1 Score: 0.8303030303030302

Confusion Matrix:

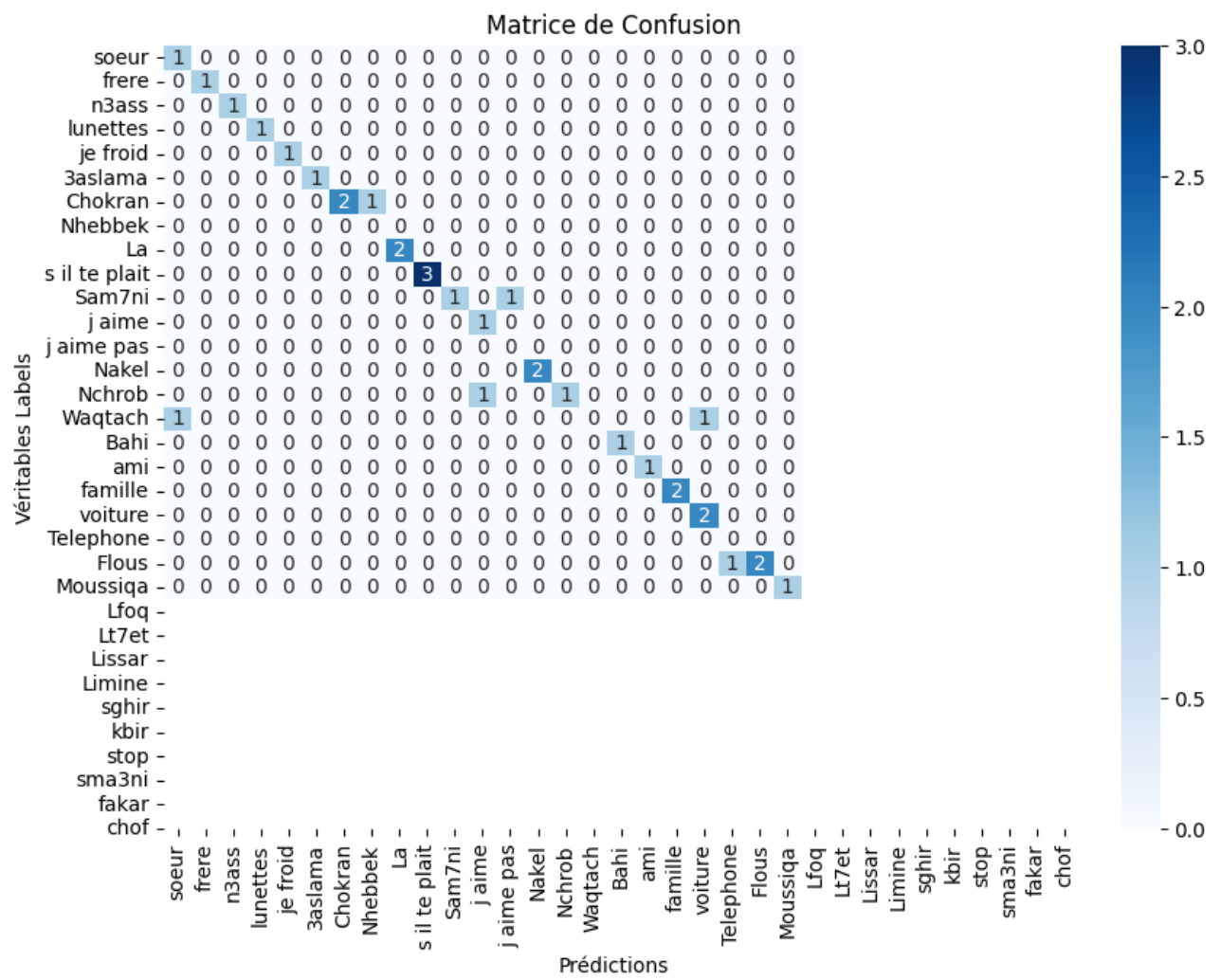


FIGURE 4.2 – Confusion Matrix CNN-3D

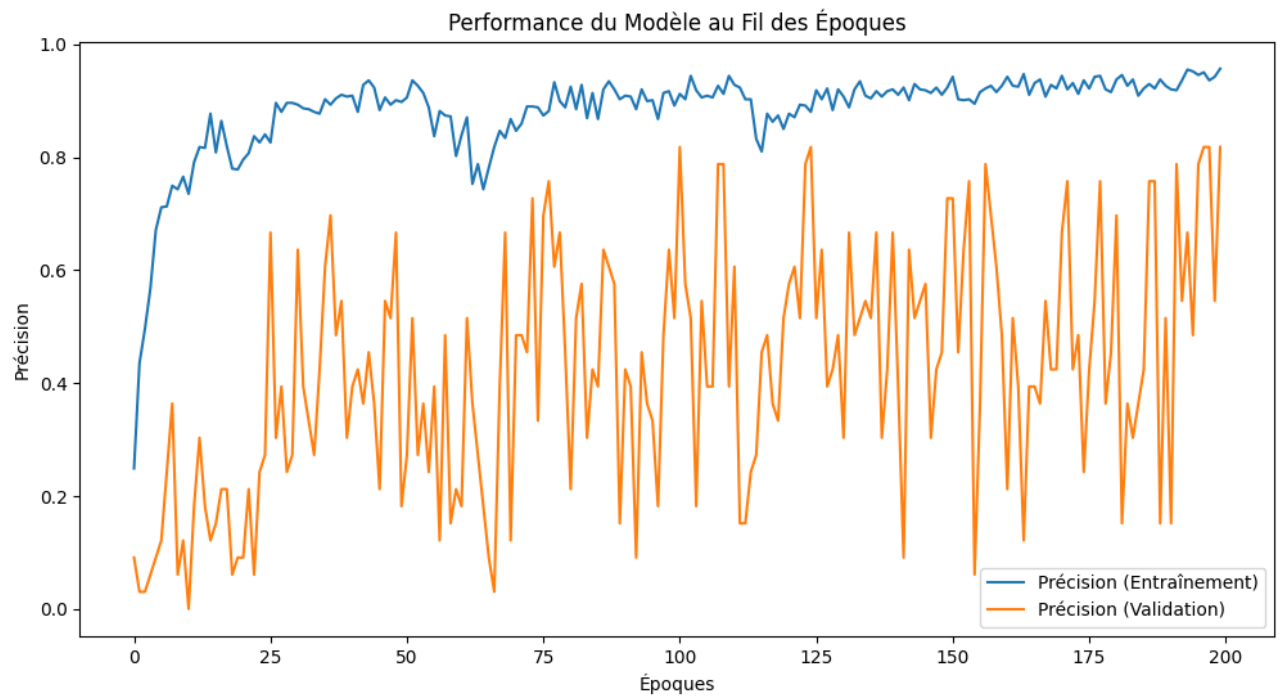


FIGURE 4.3 – Performance de CNN-3D

4.1.2 Réseaux de Neurones Universels (RNU)

Les réseaux neuronaux mondiaux (RNU) explorent des approches alternatives de la reconnaissance et de la classification des gestes. Ces réseaux offrent une grande flexibilité dans l'apprentissage des représentations gestuelles en combinant différentes structures et techniques d'apprentissage automatique. En intégrant des mécanismes d'apprentissage adaptatif, les RSU peuvent saisir les propriétés complexes et variées des gestes, améliorant ainsi la capacité du modèle à généraliser et à s'adapter aux différences dans les données gestuelles. Cette flexibilité permet aux RUN de traiter efficacement des données avec des propriétés hétérogènes et d'améliorer la précision de la reconnaissance gestuelle dans différents contextes.

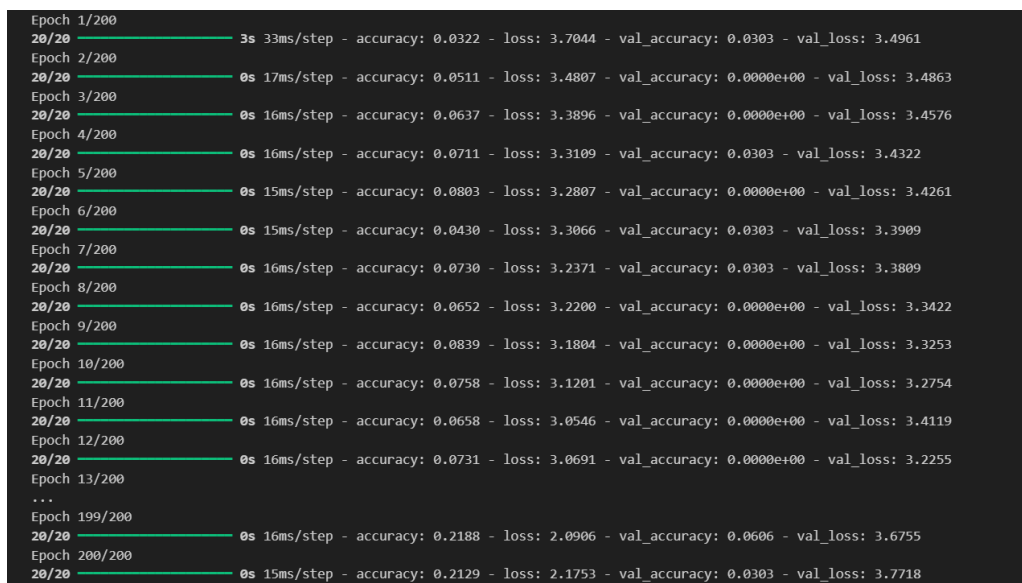


FIGURE 4.4 – RNU Training

Évaluer la Performance du Modèle :

Accuracy: 0.06060606060606061

Precision: 0.06060606060606061

Recall (Sensitivity): 0.06060606060606061

F1 Score: 0.06060606060606061

Confusion Matrix:

4.1.3 Long Short-Term Memory (LSTM)

La mémoire à court terme (LSTM) est une catégorie spécialisée de réseaux neuronaux récurrents (RNNs) conçue pour traiter et prédire des séquences de données en capturant les dépendances temporelles à long terme. Les LSTM, développés pour surmonter les contraintes des réseaux NN traditionnels, offrent une solution efficace aux problèmes d'explosion progressive et de disparition qui peuvent se produire dans les réseaux de formation sur de longues séquences.

La structure des LSTM se compose de cellules de mémoire et de trois types de portes :

- **Porte d'oubli (Forget Gate)** : Décide des informations à supprimer de la cellule mémoire.
- **Porte d'entrée (Input Gate)** : Contrôle quelles nouvelles informations doivent être ajoutées à la cellule de mémoire.
- **Porte de sortie (Output Gate)** : Détermine quelles informations de la cellule de mémoire doivent être sorties et utilisées pour la prédiction.

Ces portes permettent aux LSTM de conserver et d'oublier les informations de manière flexible, ce qui les rend particulièrement efficaces pour des tâches telles que la reconnaissance de séquences, la traduction automatique et la modélisation de séries temporelles.

- Application aux Gestes

Dans le contexte de la reconnaissance gestuelle, les LSTM sont utilisés pour modéliser la séquence des mouvements du langage des signes. En capturant les dépendances temporelles entre les gestes et le traitement des séquences gestuelles sous forme de données temporelles, les LSTM améliorent la précision des modèles de reconnaissance gestuelle, en tenant compte de la dynamique du mouvement dans le temps. Ceci est particulièrement important dans les applications où les gestes sont continus et peuvent varier considérablement au fil du temps, ce qui permet une compréhension plus complète et précise des gestes observés.

```

Epoch 1/215
20/20 — 6s 38ms/step - categorical_accuracy: 0.0432 - loss: 3.5591
Epoch 2/215
20/20 — 1s 48ms/step - categorical_accuracy: 0.0250 - loss: 3.5045
Epoch 3/215
20/20 — 1s 50ms/step - categorical_accuracy: 0.0943 - loss: 3.4533
Epoch 4/215
20/20 — 1s 54ms/step - categorical_accuracy: 0.0470 - loss: 3.4430
Epoch 5/215
20/20 — 1s 46ms/step - categorical_accuracy: 0.0780 - loss: 3.1748
Epoch 6/215
20/20 — 1s 42ms/step - categorical_accuracy: 0.1137 - loss: 2.9041
Epoch 7/215
20/20 — 1s 39ms/step - categorical_accuracy: 0.1210 - loss: 2.7907
Epoch 8/215
20/20 — 1s 38ms/step - categorical_accuracy: 0.1104 - loss: 2.7076
Epoch 9/215
20/20 — 1s 40ms/step - categorical_accuracy: 0.1432 - loss: 2.4950
Epoch 10/215
20/20 — 1s 40ms/step - categorical_accuracy: 0.2102 - loss: 2.3524
Epoch 11/215
20/20 — 1s 39ms/step - categorical_accuracy: 0.2203 - loss: 2.3820
Epoch 12/215
20/20 — 1s 39ms/step - categorical_accuracy: 0.2539 - loss: 2.2178
Epoch 13/215
...
Epoch 214/215
20/20 — 1s 41ms/step - categorical_accuracy: 0.9810 - loss: 0.0558
Epoch 215/215
20/20 — 1s 41ms/step - categorical_accuracy: 0.9538 - loss: 0.1731

```

FIGURE 4.7 – Entrainement de LSTM

Évaluer la Performance du Modèle :

Accuracy: 0.8484848484848485

Precision: 0.8434343434343434

Recall (Sensitivity): 0.8484848484848485

F1 Score: 0.8333333333333334

Confusion Matrix:

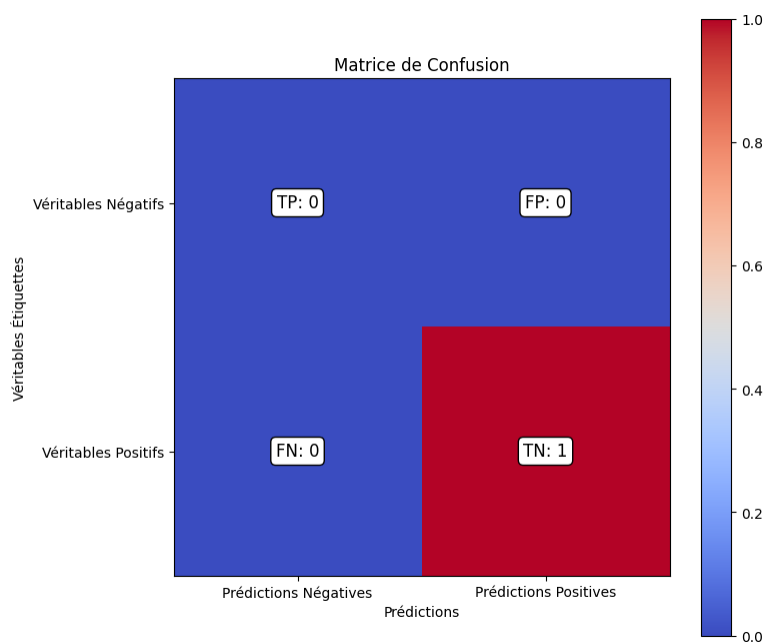


FIGURE 4.8 – Confusion Matrix LSTM

4.1.4 Random Forest

Le **Random Forest** est une méthode d'apprentissage automatique utilisée pour la classification et la régression. Elle repose sur la construction d'une forêt d'arbres de décision et combine leurs résultats pour améliorer la précision et la robustesse des prévisions.

- **Principe** : La méthode utilise plusieurs arbres de décision, chacun formé à partir d'un échantillon aléatoire des données d'entraînement. Cette approche, appelée *bagging* (bootstrap aggregating), réduit la variabilité et améliore la généralisation du modèle.
- **Construction des Arbres** :
 - o **Échantillonnage** : Chaque arbre est construit à partir d'un échantillon bootstrap des données d'entraînement.
 - o **Sélection des Caractéristiques** : À chaque nœud, un sous-ensemble aléatoire de caractéristiques est sélectionné pour déterminer la meilleure division, ce qui réduit la corrélation entre les arbres.
 - o **Construction** : Les arbres sont construits en divisant les données à chaque nœud jusqu'à atteindre un critère d'arrêt.
- **Prédiction** :
 - o **Classification** : Chaque arbre vote pour une classe, et la classe avec le plus grand nombre de votes est sélectionnée.
 - o **Régression** : La prédiction est la moyenne des prédictions de tous les arbres.
- **Avantages** :
 - o **Robustesse** : Moins susceptible au surajustement par rapport aux arbres de décision uniques.
 - o **Gestion des Données Manquantes** : Peut gérer les données manquantes en utilisant les prédictions des autres arbres.
 - o **Importance des Variables** : Fournit une mesure de l'importance des caractéristiques, utile pour l'interprétation du modèle.
- **Évaluation de la Performance** :

Accuracy: 0.8182

Precision: 0.8788

Recall: 0.8182

F1 Score: 0.8384

Confusion Matrix:

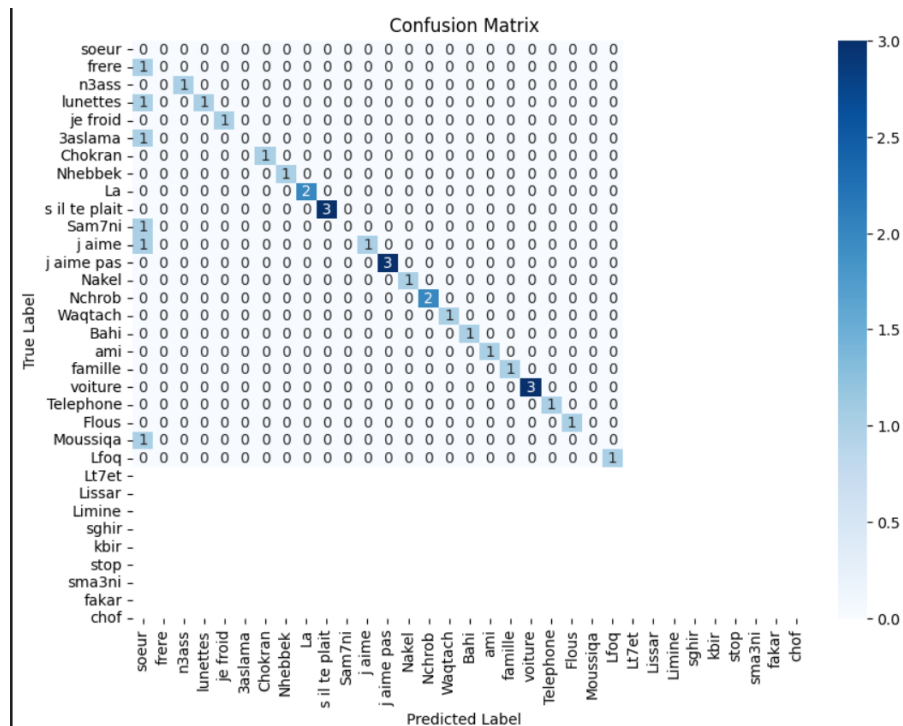


FIGURE 4.9 – Confusion Matrix RandomForest

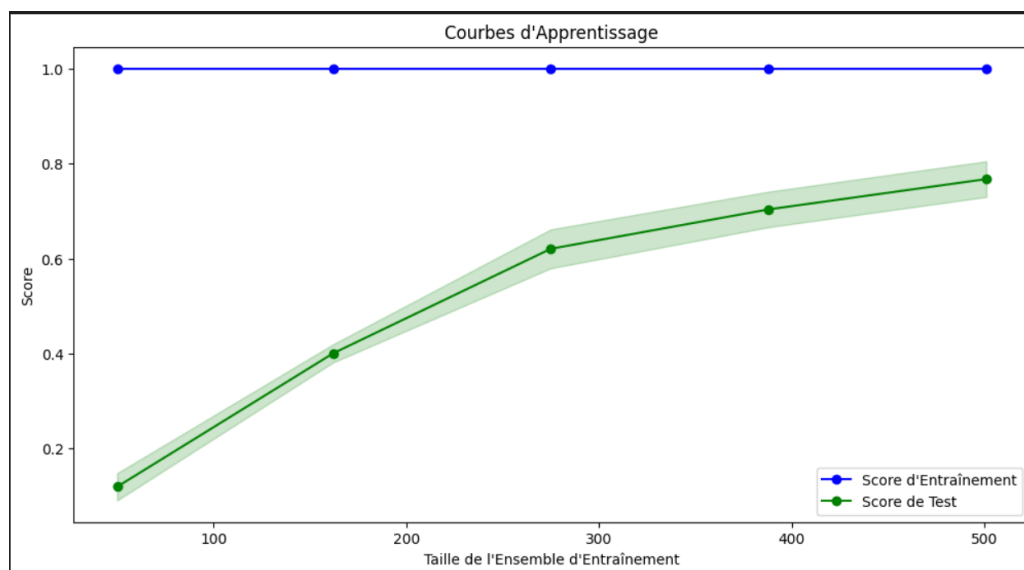


FIGURE 4.10 – Visualisation de Performance RandomForest

4.1.5 Machines à Vecteurs de Support (SVM)

Les Machines à Vecteurs de Support (SVM, *Support Vector Machines*) sont une méthode efficace de classification et de régression, en particulier pour les problèmes de classification bilatéraux. Le but du MVS est de trouver un aéronef excessif qui sépare les données en catégories tout en maximisant la marge entre ces catégories. La marge est définie comme étant la distance entre l'aéronef surséparé et les exemples les plus proches de chaque catégorie, appelés vecteurs de support.

Pour un problème de classification binaire, considérons un ensemble de données $\{(x_i, y_i)\}_{i=1}^n$, où $x_i \in \mathbb{R}^d$ est un vecteur de caractéristiques et $y_i \in \{-1, +1\}$ est l'étiquette de classe. L'objectif des SVM est de déterminer un hyperplan $\mathbf{w} \cdot \mathbf{x} + b = 0$ qui sépare les deux classes avec la plus grande marge possible. Ce problème peut être formulé comme suit :

$$\begin{aligned} &\text{Maximiser} \quad \frac{2}{\|\mathbf{w}\|} \\ &\text{Sous les contraintes} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall i \end{aligned} \tag{4.1}$$

où $\mathbf{w}$ est le vecteur normal à l'hyperplan et b est le terme de biais.

Lorsque les données ne sont pas détachables par écrit, le SVM utilise les fonctions de base pour convertir les données en un espace de dimensions supérieures où un avion linéaire excessif peut être trouvé. Les fonctions de base permettent le calcul du produit standard dans cet espace sans faire le changement explicitement. Certains des noyaux actuels incluent :

- **Noyau Linéaire** : $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$
- **Noyau Polynomial** : $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + c)^d$
- **Noyau Gaussien (RBF)** : $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$

Les principaux hyperparamètres des SVM sont :

- **Paramètre de Régularisation C** : Ce paramètre contrôle le compromis entre maximiser la marge et minimiser l'erreur de classification. Un C élevé favorise une classification parfaite, tandis qu'un C faible permet une marge plus large avec une certaine erreur.
- **Paramètre du noyau** : Ce paramètre dépend du noyau utilisé, comme d dans le noyau polynomial ou σ dans le noyau RBF.

Les SVM sont particulièrement appréciées pour leur capacité à gérer des espaces de grande dimension et leur flexibilité à résoudre des problèmes non linéaires grâce aux fonctions noyaux.

Évaluer la Performance du Modèle :

Accuracy: 0.9393939393939394

Precision: 0.9696969696969697

Recall (Sensitivity): 0.9393939393939394

F1 Score: 0.9494949494949495

Confusion Matrix:

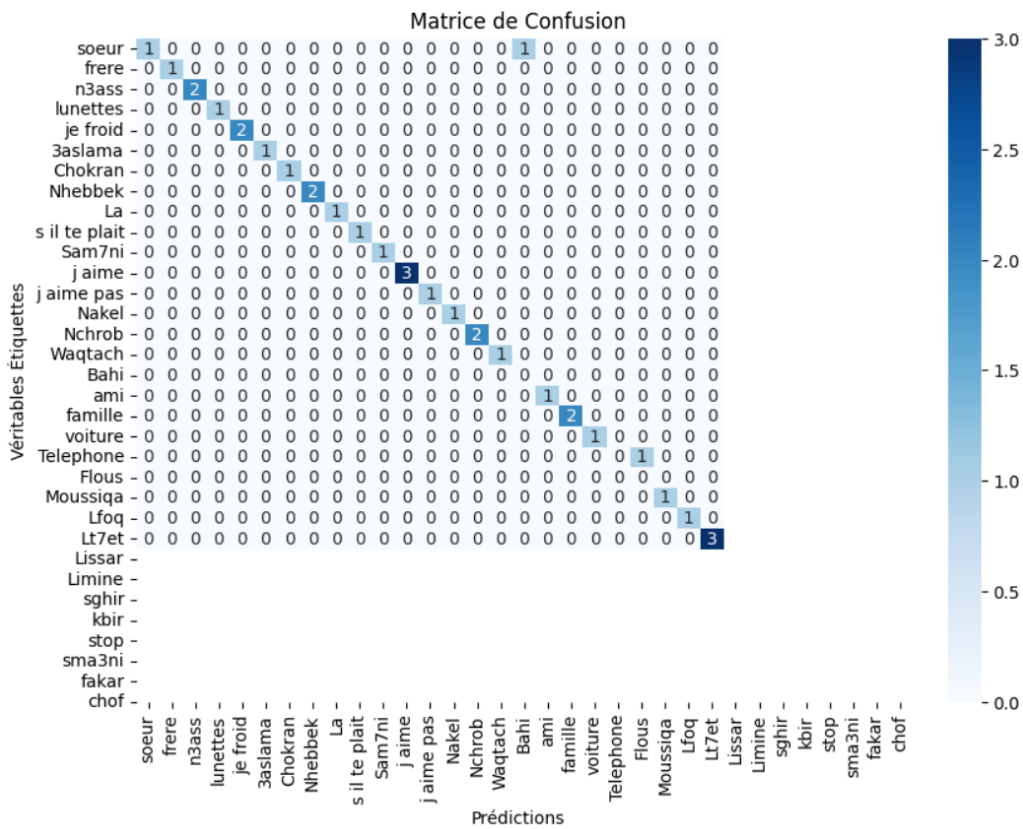


FIGURE 4.11 – Confusion Matrix SVM

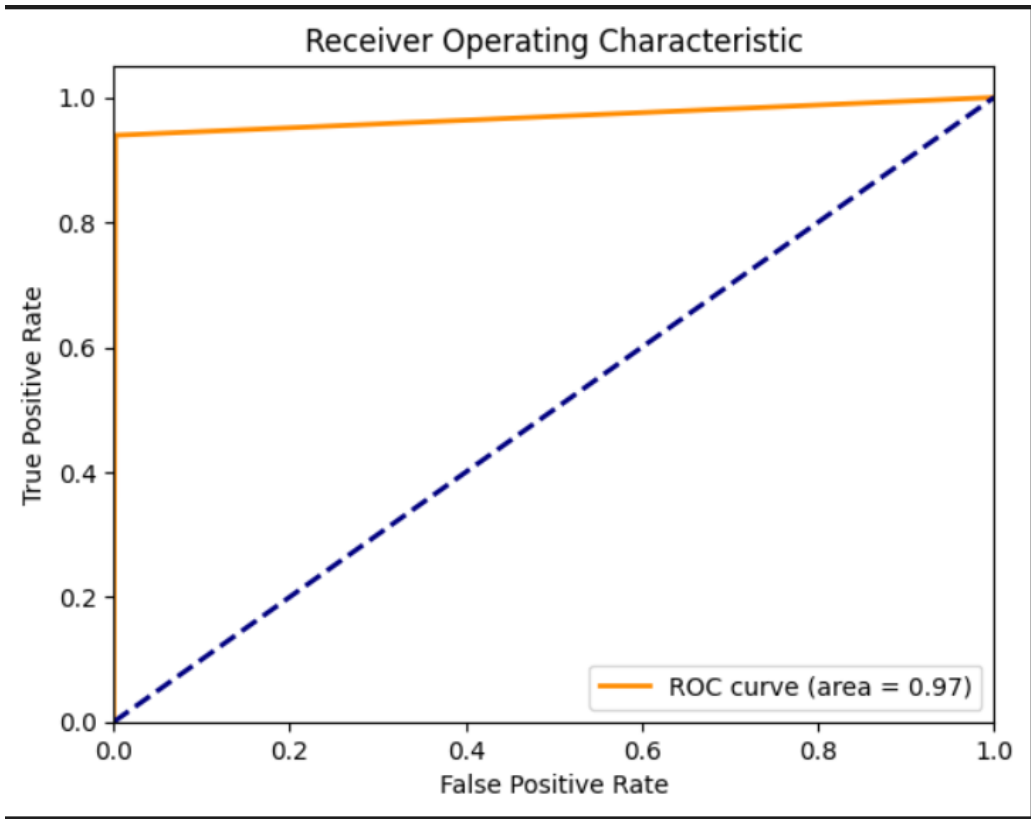


FIGURE 4.12 – Visualisation de Performance SVM

4.1.6 Combinaisons de Modèles

4.1.6.1 LSTM + CNN

Cette approche combine les **CNN** (Convolutional Neural Networks) et les **LSTM** (Long Short-Term Memory) pour tirer parti des forces de chaque type de réseau. Les **CNN** sont utilisés pour extraire des caractéristiques spatiales importantes à partir d'images ou de séquences d'images en appliquant des filtres convolutifs. Ces caractéristiques sont ensuite fournies aux **LSTM**, qui capturent les dynamiques temporelles et les dépendances à long terme dans les séquences de données. Cette combinaison permet d'analyser efficacement à la fois les aspects visuels et temporels des données, améliorant ainsi la précision des tâches telles que la reconnaissance de gestes ou la prédiction

```
Epoch 1/80
10/10 — 3s 64ms/step - accuracy: 0.0366 - loss: 3.5234 - val_accuracy: 0.0000e+00 - val_loss: 3.4624
Epoch 2/80
10/10 — 0s 28ms/step - accuracy: 0.0419 - loss: 3.4859 - val_accuracy: 0.0909 - val_loss: 3.4185
Epoch 3/80
10/10 — 0s 25ms/step - accuracy: 0.0636 - loss: 3.4109 - val_accuracy: 0.0909 - val_loss: 3.3363
Epoch 4/80
10/10 — 0s 26ms/step - accuracy: 0.1040 - loss: 3.2896 - val_accuracy: 0.0606 - val_loss: 3.3059
Epoch 5/80
10/10 — 0s 27ms/step - accuracy: 0.1269 - loss: 3.1136 - val_accuracy: 0.0909 - val_loss: 3.1662
Epoch 6/80
10/10 — 0s 26ms/step - accuracy: 0.1708 - loss: 2.9345 - val_accuracy: 0.0606 - val_loss: 2.9708
Epoch 7/80
10/10 — 0s 24ms/step - accuracy: 0.2253 - loss: 2.7591 - val_accuracy: 0.1212 - val_loss: 2.7324
Epoch 8/80
10/10 — 0s 26ms/step - accuracy: 0.2140 - loss: 2.5841 - val_accuracy: 0.2424 - val_loss: 2.5491
Epoch 9/80
10/10 — 0s 26ms/step - accuracy: 0.2709 - loss: 2.4199 - val_accuracy: 0.2727 - val_loss: 2.3241
Epoch 10/80
10/10 — 0s 24ms/step - accuracy: 0.3348 - loss: 2.2276 - val_accuracy: 0.4545 - val_loss: 2.1650
Epoch 11/80
10/10 — 0s 27ms/step - accuracy: 0.3689 - loss: 2.0828 - val_accuracy: 0.3030 - val_loss: 2.1624
Epoch 12/80
10/10 — 0s 25ms/step - accuracy: 0.3900 - loss: 1.9711 - val_accuracy: 0.5455 - val_loss: 1.8891
Epoch 13/80
...
Epoch 80/80
10/10 — 0s 28ms/step - accuracy: 0.8942 - loss: 0.3515 - val_accuracy: 0.8485 - val_loss: 0.5964
2/2 — 0s 5ms/step - accuracy: 0.8469 - loss: 0.6017
Test Accuracy: 0.85
```

FIGURE 4.13 – CNN+LSTM Training

Évaluer la Performance du Modèle :

Accuracy: 0.94

Precision: 0.90

Recall (Sensitivity): 0.94

F1 Score: 0.92

Confusion Matrix:

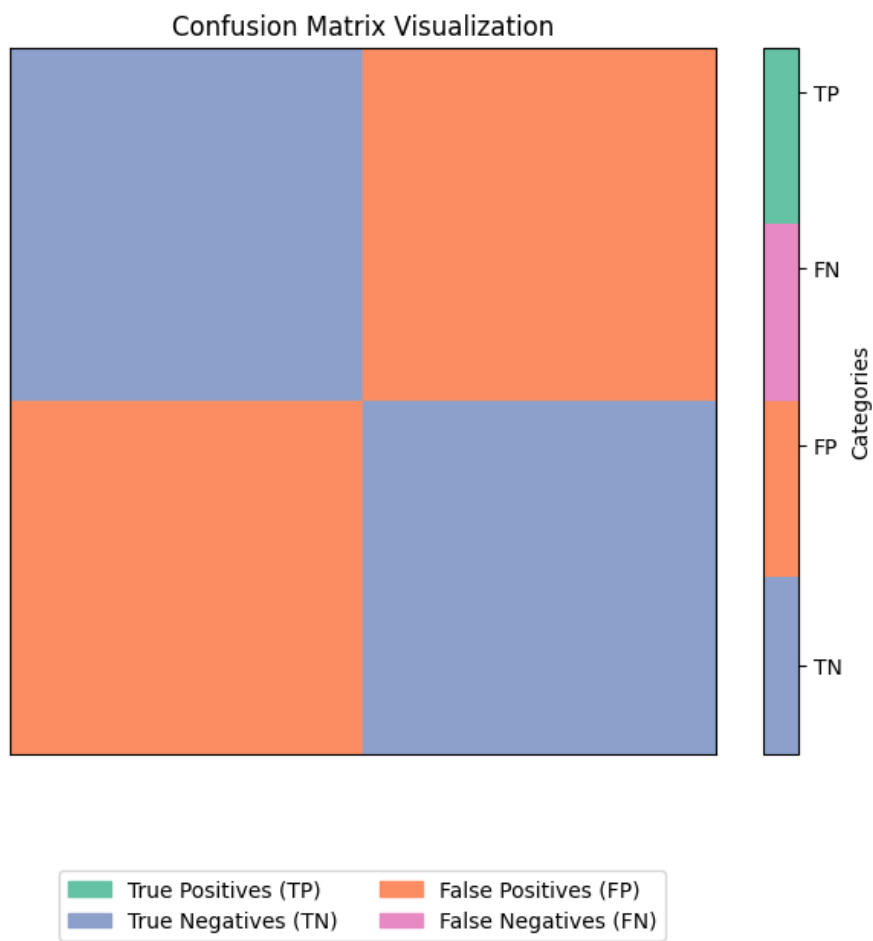


FIGURE 4.14 – Confusion Matrix CNN+LSTM

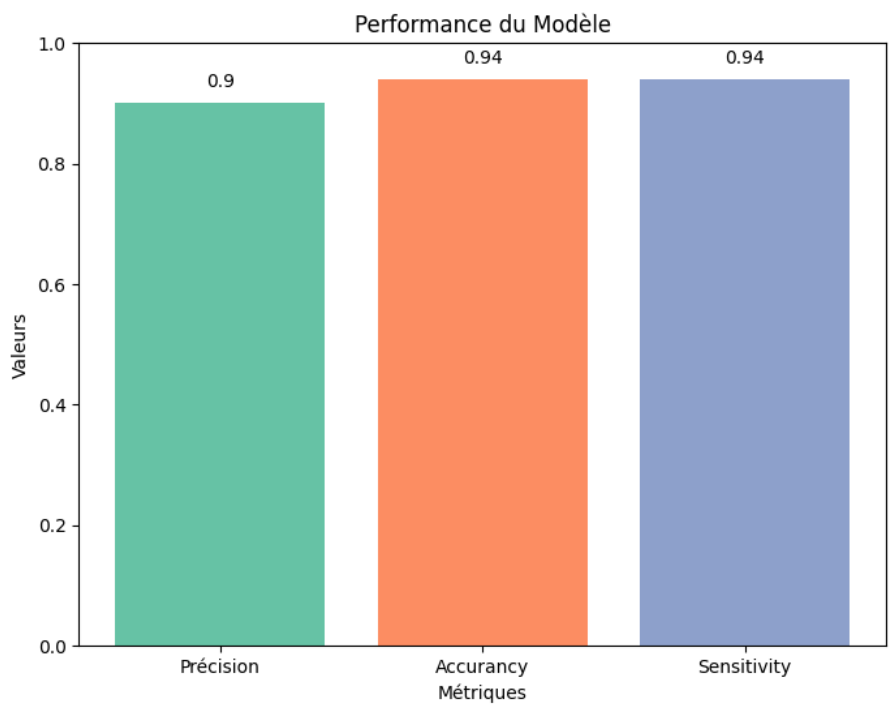


FIGURE 4.15 – Visualisation de Performance CNN +LSTM

4.1.6.2 LSTM + Random Forest

Le modèle **LSTM (Long Short-Term Memory)** est une structure de réseau neuronal répétitive conçue pour apprendre les dépendances temporelles à long terme. Contrairement aux réseaux neuronaux récurrents traditionnels (RNN), les LSTM utilisent des cellules de mémoire et des portes pour réguler l'information au fil du temps. Cela permet au modèle de capturer des relations complexes dans les séquences de données, ce qui est particulièrement utile pour les tâches impliquant des séquences temporelles ou des séquences temporelles. La fonction de transfert du mécanisme peut être formulée comme suit :

$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\
 \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\
 C_t &= f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\
 h_t &= o_t \cdot \tanh(C_t)
 \end{aligned} \tag{4.2}$$

où f_t , i_t , o_t sont les vecteurs de portes de l'oubli, d'entrée, et de sortie respectivement, C_t est la cellule de mémoire, et h_t est la sortie du LSTM.

Le **Random Forest** est un modèle d'ensemble basé sur des arbres de décision. Il construit plusieurs arbres de décision durant l'apprentissage et combine leurs prédictions pour améliorer la précision et éviter le surapprentissage. La prédiction d'un Random Forest est la moyenne des prédictions des arbres individuels ou la classe majoritaire dans le cas de la classification. Le modèle Random Forest peut être décrit par l'équation :

$$\hat{y} = \frac{1}{N} \sum_{n=1}^N T_n(x) \tag{4.3}$$

où $\hat{y}$ est la prédiction, N est le nombre d'arbres dans la forêt, et $T_n(x)$ est la prédiction de l'arbre n pour l'entrée x .

Justification de la Combinaison

La combinaison de LSTM et de Random Forest peut tirer parti des forces des deux modèles. Le LSTM est efficace pour capturer les dépendances temporelles et les séquences complexes, tandis que la forêt aléatoire est forte contre l'apprentissage excessif et peut gérer des interactions non linéaires complexes entre les fonctionnalités.

L'idée est d'utiliser LSTM pour extraire les propriétés temporelles des données en série, puis d'utiliser ces propriétés comme entrée dans le modèle de forêt aléatoire. Cette approche permet à la forêt aléatoire de tirer parti des riches représentations temporelles fournies par LSTM tout en exploitant sa capacité à effectuer une classification précise.

Approche d'Ensemble

L'approche globale consiste à combiner les résultats de plusieurs modèles pour obtenir une prévision finale. Dans le cas de LSTM et de la forêt aléatoire, une façon possible est d'utiliser la LSTM pour générer des caractéristiques temporelles et d'utiliser ces propriétés comme entrée dans le modèle de la forêt aléatoire. Cela peut être formalisé comme suit :

1. **Extraction des Caractéristiques** : Utiliser le LSTM pour transformer les données séquentielles en un vecteur de caractéristiques temporelles.
2. **Classification** : Passer ce vecteur à travers un modèle Random Forest pour obtenir la prédiction finale.

Cette méthode peut améliorer les performances du modèle en combinant les avantages des deux architectures.

Conclusion

La combinaison de LSTM et de Random Forest est une approche efficace des problèmes liés aux données consécutives, qui tire parti des capacités de capture des LSTM en fonction du temps et de la puissance des forêts aléatoires contre l'apprentissage excessif. Cette approche d'agrégation permet une meilleure modélisation des données complexes et améliore la précision des prédictions.

Évaluer la Performance du Modèle :

Accuracy: 0.6363636363636364

Precision: 0.7222222222222222

Recall (Sensitivity): 0.6363636363636364

F1 Score: 0.6393939393939393

Confusion Matrix:

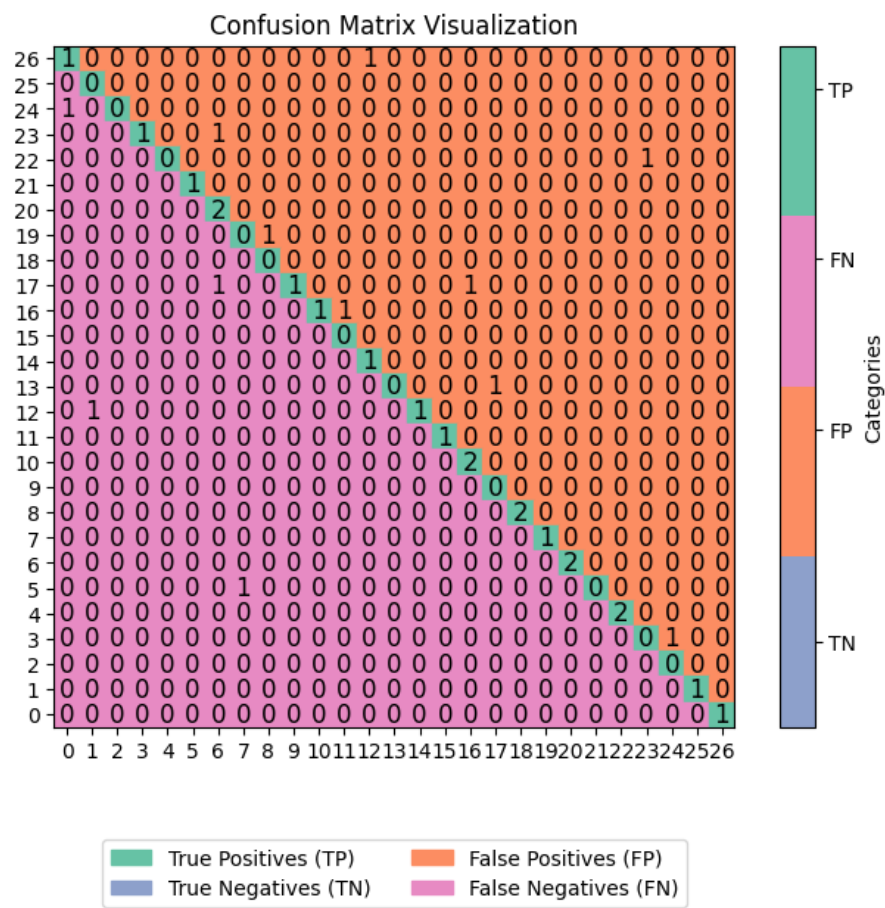


FIGURE 4.16 – Confusion Matrix RandomForest+LSTM

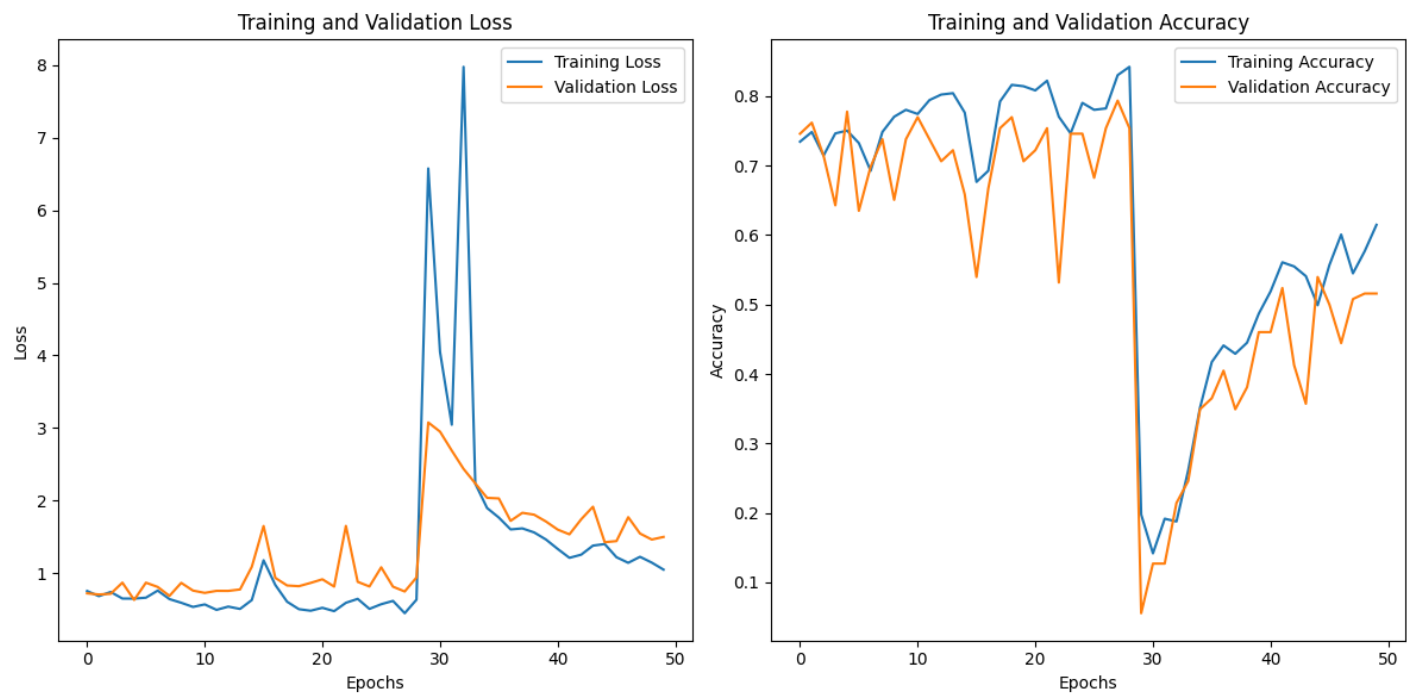


FIGURE 4.17 – Visualisation de Performance de RandomForest +LSTM

4.1.6.3 LSTM + CNN + Random Forest

La combinaison des modèles **LSTM (Long Short-Term Memory)**, **CNN (Convolutional Neural Networks)**, et **Random Forest** offre une approche puissante pour traiter des données complexes, intégrant à la fois des aspects temporels et spatiaux.

- **LSTM** : Spécialisé dans le traitement des séquences temporelles en capturant des dépendances à long terme.
- **CNN** : Conçu pour extraire des caractéristiques spatiales à partir d'images ou de séquences structurées.
- **Random Forest** : Utilise un ensemble d'arbres de décision pour améliorer la précision et réduire l'apprentissage excessif.

Justification de la Combinaison : LSTM et CNN traitent respectivement les aspects temporels et spatiaux des données, tandis que Random Forest exploite les caractéristiques extraites pour des prédictions robustes.

- **LSTM + CNN** : Permet de traiter les données avec des structures temporelles et spatiales.
- **CNN + Random Forest** : Les caractéristiques extraites par CNN servent d'entrée à Random Forest, améliorant ainsi les prédictions.

Étapes de la Combinaison :

- **Extraction des Caractéristiques** : CNN extrait des caractéristiques spatiales, qui sont ensuite traitées par LSTM pour modéliser les dépendances temporelles.
- **Classification/Prédiction** : Les sorties du LSTM sont utilisées comme entrée pour Random Forest, qui effectue les prédictions finales.

Conclusion : Cette combinaison permet de traiter efficacement des données complexes, intégrant des aspects spatiaux et temporels pour améliorer les performances dans des tâches telles que la reconnaissance d'images chronologiques.

Évaluation du Modèle :

Accuracy : 0.8485

Precision: 0.8737

Recall: 0.8485

F1 Score: 0.8450

Confusion Matrix:

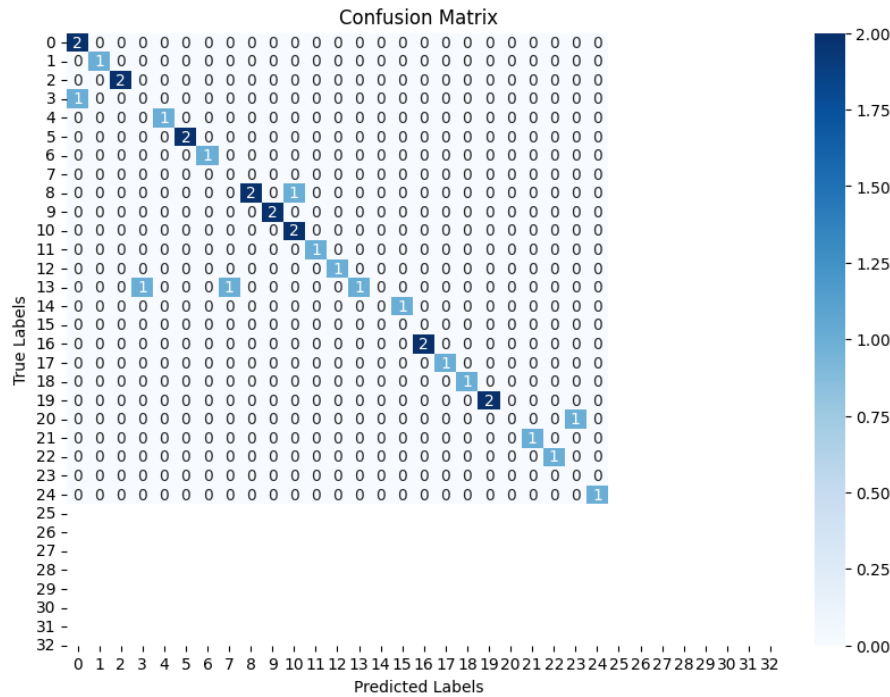


FIGURE 4.18 – Matrice de Confusion CNN+RandomForest+LSTM

4.2 Comparaison et Sélection du Modèle

4.2.1 Comparaison du Modèle

Pour développer une application mobile avec Flutter pour la traduction du langage des signes en texte et en message vocal, l'étape cruciale a été de comparer et de choisir des modèles d'apprentissage automatique pour reconnaître les gestes. Cette phase est essentielle pour assurer une performance optimale de l'application en termes d'exactitude et de réactivité, affectant directement l'expérience utilisateur et l'efficacité de la traduction.

Pour répondre aux besoins de ce projet, de nombreux modèles ont été testés, chacun avec ses propres avantages et limites :

- **Random Forest (RF)** : Ce modèle a été évalué pour sa capacité à classer les gestes après extraction des propriétés. Malgré son efficacité dans les tâches de classification en général, sa performance dans ce contexte était limitée par son incapacité à saisir les dépendances temporelles des gestes.
- **Réseaux de Neurones Convolutifs (CNN)** : Les CNN ont été utilisés pour extraire des propriétés spatiales des gestes, permettant une meilleure compréhension des formes et des motifs dans les images gestuelles. Faire preuve d'une bonne performance pour reconnaître les gestes individuels.

- **Long Short-Term Memory (LSTM)** : Des LSTM ont été mis en œuvre pour gérer les séquences temporelles des gestes, capturant la dynamique temporelle nécessaire à une identification précise. Ce modèle s'est avéré efficace pour traiter les séquences de gestes continus.
- **SVM (Support Vector Machine)** : Bien que les SVM aient réussi dans certains cas de classification, ils n'ont pas surpassé d'autres modèles dans le contexte de la reconnaissance gestuelle en raison de leur gestion limitée des séquences complexes.
- **Combinaisons de Modèles** :
 - o **Random Forest + CNN** : Cette combinaison a permis l'utilisation des propriétés spatiales extraites des CNN et les capacités de notation solides de la forêt aléatoire.
 - o **Random Forest + LSTM** : L'association des forêts aléatoires avec les LSTM a permis de combiner la classification efficace et la gestion du séquençage temporel.
 - o **CNN + LSTM** : Ce modèle a tiré parti des points forts des CNN pour extraire les caractéristiques spatiales et les LSTM pour les dépendances temporelles, démontrant une amélioration significative par rapport aux modèles individuels.
 - o **Random Forest + CNN + LSTM** : La combinaison de ces trois approches a donné des résultats plus robustes, tirant parti des forces de chaque modèle afin d'identifier les gestes avec plus de précision.

Models	Accuracy	Precision	Sensitivity	F1 Score
LSTM	0.8484	0.8434	0.8484	0.8333
CNN	0.81818181	0.8888	0.81818	0.83030
RNU	0.0606	0.06060	0.06060	0.06060
RandomForest	0.8182	0.87878	0.81818	0.8383
SVM	0.9393	0.9696	0.9393	0.9494
LSTM+CNN	0.94	0.90	0.94	0.92
LSTM+RandomForst	0.63636	0.7222	0.63636	0.6393
LSTM+CNN+RandomForest	0.8484	0.8737	0.84848	0.8450

TABLE 4.1 – Comparaison Des Modèles

L'évaluation des performances des différents modèles révèle des différences notables dans leur efficacité pour la tâche de classification. Les résultats, détaillés dans le Tableau 4.1, montrent que les performances varient considérablement entre les modèles, ce qui influence le choix du modèle optimal pour notre application.

- **LSTM** : Avec une précision de 0.8484 et un score F1 de 0.8333, le modèle LSTM montre une bonne capacité à traiter des données temporelles, mais il n'est pas le meilleur en termes de précision globale et de score F1 par rapport aux autres modèles combinés.

- **CNN** : Le modèle CNN obtient une précision de 0.8182 et un score F1 de 0.8303. Il excelle dans la précision, mais a une performance légèrement inférieure en termes de sensibilité et d'exactitude globale par rapport à certains modèles combinés.
- **Random Forest** : Ce modèle a une précision de 0.8182 et un score F1 de 0.8383. Bien qu'efficace, il n'atteint pas les performances des meilleures combinaisons de modèles.
- **SVM** : Le modèle SVM obtient les meilleures performances avec une précision de 0.9393, un score F1 de 0.9494, et une sensibilité de 0.9393. Cependant, son utilisation peut être limitée par la complexité et les ressources requises.
- **LSTM + CNN** : Cette combinaison affiche des résultats impressionnants avec une précision de 0.94 et un score F1 de 0.92, montrant une excellente capacité à traiter à la fois les aspects temporels et spatiaux des données.
- **LSTM + Random Forest** : Ce modèle combiné présente des résultats moins performants, avec une précision de 0.63636 et un score F1 de 0.6393, indiquant que cette combinaison n'est pas aussi efficace que d'autres.
- **LSTM + CNN + Random Forest** : Bien que cette combinaison soit robuste avec une précision de 0.8484 et un score F1 de 0.8450, elle ne surpasse pas la combinaison LSTM + CNN en termes de performances globales.

En conclusion, bien que le modèle SVM offre les meilleures performances globales, sa complexité peut ne pas être justifiée dans tous les contextes. La combinaison **LSTM + CNN** se distingue comme le modèle optimal pour cette tâche spécifique, offrant un excellent équilibre entre précision, sensibilité et score F1. Cette combinaison permet de capturer efficacement les dépendances temporelles tout en extrayant les caractéristiques spatiales pertinentes, offrant ainsi une solution robuste et performante pour les données complexes traitées.

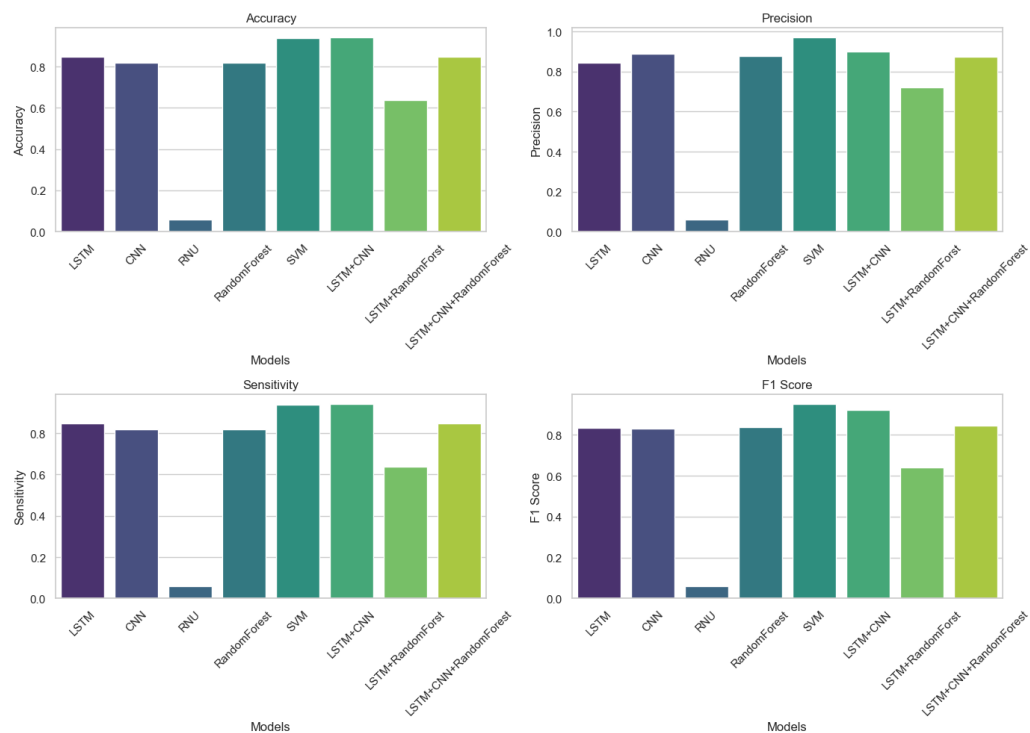


FIGURE 4.19 – Histogramme des Models

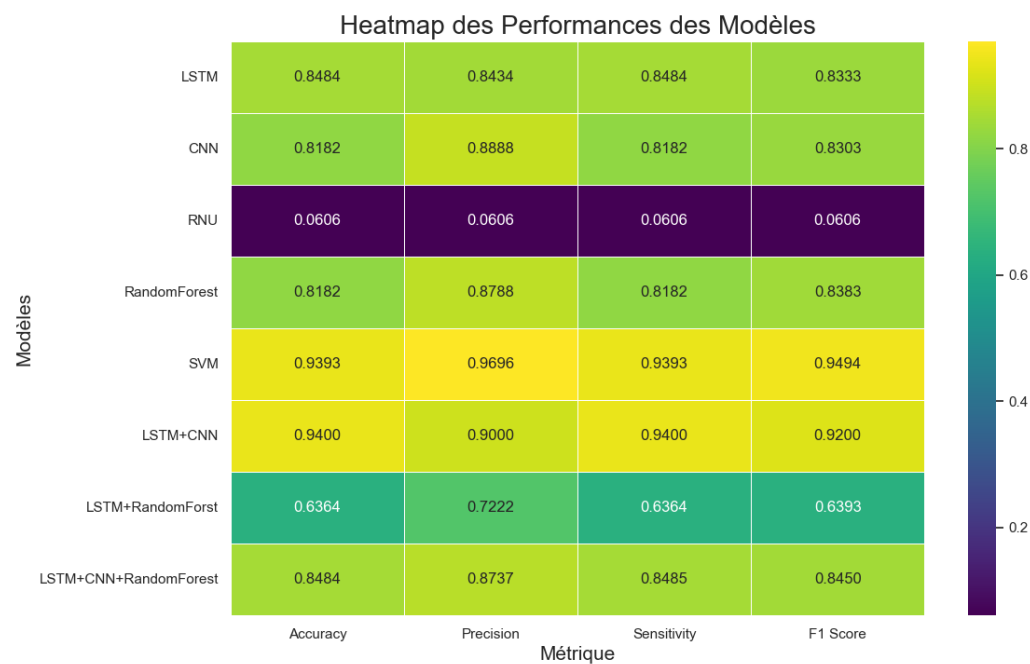


FIGURE 4.20 – Heatmap des performance des Models

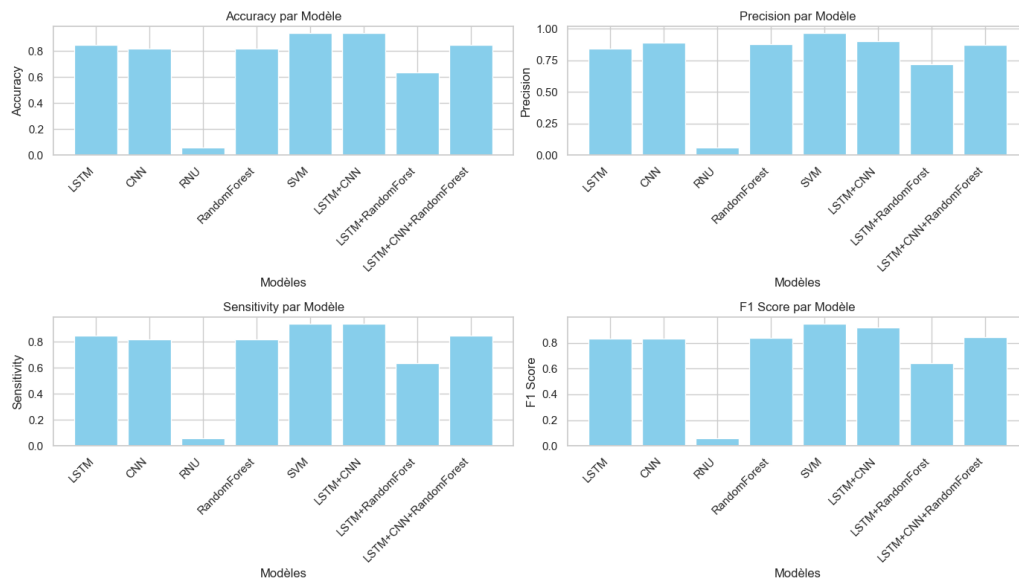


FIGURE 4.21 – Graphiques en barres

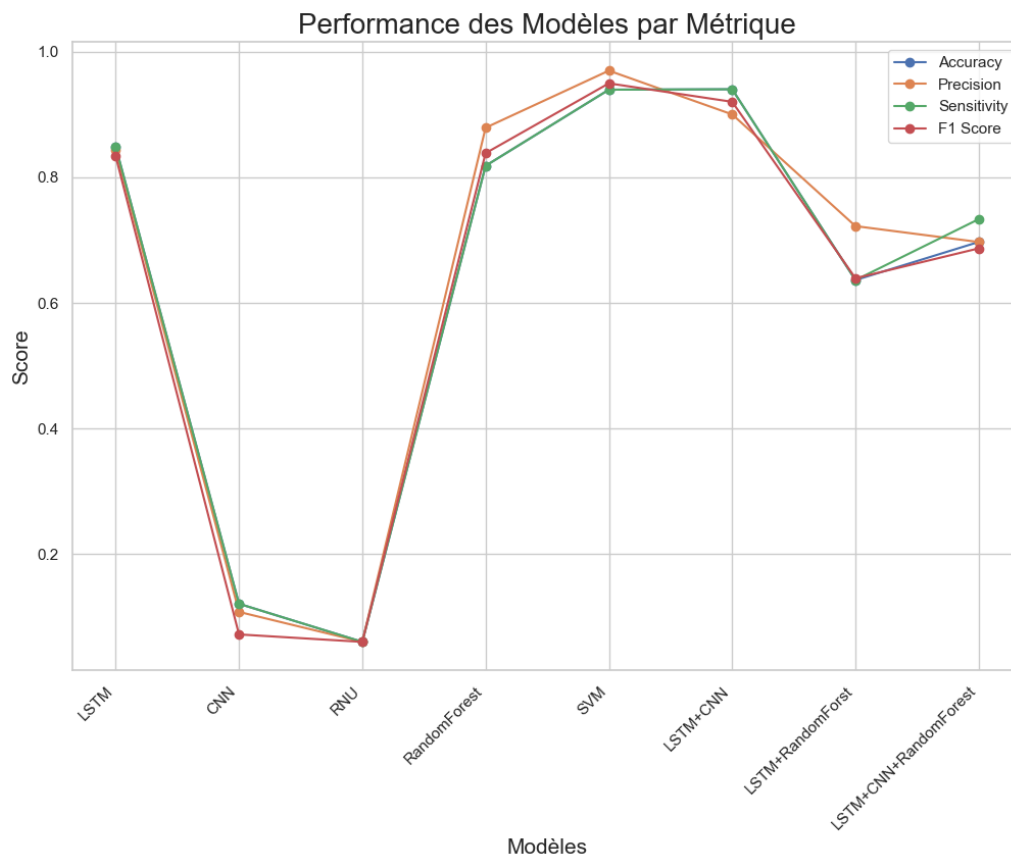


FIGURE 4.22 – Graphiques de lignes

4.2.2 Sélection du Modèle

Analyse des Modèles

Pour déterminer le meilleur modèle de la liste du tableau, il faut comparer les performances des modèles selon chaque échelle (précision, précision, sensibilité, grade F1). Le meilleur modèle peut être déterminé en fonction de l'échelle que vous jugez la plus importante pour votre tâche spécifique. Voici une analyse de chaque mesure :

1. Accuracy (Précision Globale)

L'accuracy mesure la proportion de prédictions correctes sur l'ensemble des prédictions faites. Plus l'accuracy est élevée, mieux c'est.

- **SVM** : 0.9393
- **LSTM+CNN** : 0.94
- **LSTM** : 0.8484
- **RandomForest** : 0.8182
- **LSTM+CNN+RandomForest** : 0.8485
- **LSTM+RandomForst** : 0.63636
- **CNN** : 0.8181
- **RNU** : 0.0606

Le meilleur modèle pour l'accuracy est **LSTM+CNN** avec une accuracy de 0.94.

2. Precision (Précision)

La précision mesure le pourcentage de prédictions positives réelles parmi toutes les prédictions positives. Une grande précision indique un faible taux de faux positifs.

- **SVM** : 0.9696
- **LSTM+CNN** : 0.90
- **RandomForest** : 0.87878
- **LSTM** : 0.8434
- **LSTM+CNN+RandomForest** : 0.8737373737373737
- **LSTM+RandomForst** : 0.7222

- **CNN** : 0.8888
- **RNU** : 0.06060

Le meilleur modèle pour la précision est **SVM** avec une précision de 0.9696.

3. Sensitivity (Sensibilité)

La sensibilité mesure le pourcentage de cas positifs réels détectés parmi tous les cas positifs réels. Une sensibilité élevée indique une bonne capacité à détecter les cas positifs.

- **SVM** : 0.9393
- **LSTM+CNN** : 0.94
- **LSTM** : 0.8484
- **RandomForest** : 0.81818
- **LSTM+CNN+RandomForest** : 0.7333
- **LSTM+RandomForst** : 0.63636
- **CNN** : 0.8181
- **RNU** : 0.06060

Le meilleur modèle pour la sensibilité est **LSTM+CNN** avec une sensibilité de 0.94.

4. F1 Score

Le degré de F1 est la moyenne harmonique entre précision et sensibilité. Donne un aperçu des performances du modèle en tenant compte des faux positifs et des faux négatifs.

- **SVM** : 0.9494
- **LSTM+CNN** : 0.92
- **RandomForest** : 0.8383
- **LSTM** : 0.8333
- **LSTM+CNN+RandomForest** : 0.6868
- **LSTM+RandomForst** : 0.6393
- **CNN** : 0.8303
- **RNU** : 0.06060

La comparaison des modèles présentés montre des performances variées en termes de précision, sensibilité, score F1, et exactitude. Parmi eux, le modèle **CNN+LSTM** se distingue comme étant le meilleur, affichant une précision de **0.94** et un score F1 de **0.92**, ce qui indique un bon équilibre entre la précision et la capacité de rappel.

Bien que le modèle **SVM** ait une précision légèrement supérieure (**0.9393**) et un score F1 légèrement inférieur (**0.9494**), le modèle **CNN+LSTM** combine efficacement les forces des réseaux de neurones convolutifs et des *LSTM*, offrant ainsi une performance globalement meilleure. Ce modèle est donc recommandé pour des applications nécessitant à la fois une haute précision et une robustesse générale, surpassant les autres modèles évalués dans cette comparaison. 0.9494.

Chapitre 5

Partie Développement

5.1 Architecture de l'Application Mobile Flutter

5.1.1 Introduction au Langage Dart et Projet Flutter

Le développement des applications mobiles utilisant Flutter dépend de l'utilisation du langage de programmation Dart, conçu par Google. Dart est un langage moderne orienté objet qui se distingue par sa structure simple et sa capacité d'assemblage dans le code original, ce qui le rend particulièrement adapté aux applications mobiles haute performance.

Le framework Flutter, également développé par Google, permet de créer des applications mobiles avec une interface utilisateur riche et réactive. Flutter fournit un environnement complet pour le développement d'applications mobiles, y compris une bibliothèque d'outils préconçus, un moteur d'affichage puissant et un ensemble d'outils pour l'assemblage et le débogage. Avec sa capacité à créer du code local pour Android et iOS à partir d'une seule base de code, Flutter est devenu un choix populaire pour le développement d'applications multiplateformes.

Dans ce projet, nous avons utilisé Flutter pour concevoir l'application de reconnaissance gestuelle. Cette application est basée sur une structure modulaire qui facilite l'intégration des composants de l'interface utilisateur, la gestion des états et la communication avec le backend de Flask. Utiliser Flutter nous a permis de tirer parti du développement rapide et de l'expérience utilisateur cohérente sur différentes plateformes mobiles.

5.1.1.1 Structure du Projet Flutter

La structure du projet Flutter suit une organisation unifiée qui facilite le développement modulaire et évolutif d'applications mobiles. Aux racines du projet, on retrouve des fichiers clés tels que `lib/`, `android/`, `ios/`, `assets/`, ainsi que les fichiers de configuration comme `pubspec.yaml`.

Le dossier `lib/` contient la majeure partie du code source de l'application, y compris les fichiers Dart qui définissent les widgets, les classes, les services, et la logique métier. Ce dossier est souvent subdivisé en plusieurs sous-dossiers, tels que `ui/` pour les interfaces utilisateur, `models/` pour les modèles de données, et `services/` pour les services et la gestion des API.

Les dossiers `android/` et `ios/` contiennent des configurations spécifiques à la plateforme, vous permettant de personnaliser et d'assembler l'application pour Android et iOS, respectivement. Le fichier `pubspec.yaml` est crucial pour gérer les dépendances, ressources et configurations globales du projet. Il permet de faire la publicité des packages externes utilisés, tels que les bibliothèques d'outils ou les accessoires nécessaires à l'application.

Cette organisation modulaire permet de maintenir un code propre et réutilisable, facilitant la collaboration entre les développeurs et ajoutant de nouvelles fonctions au projet.

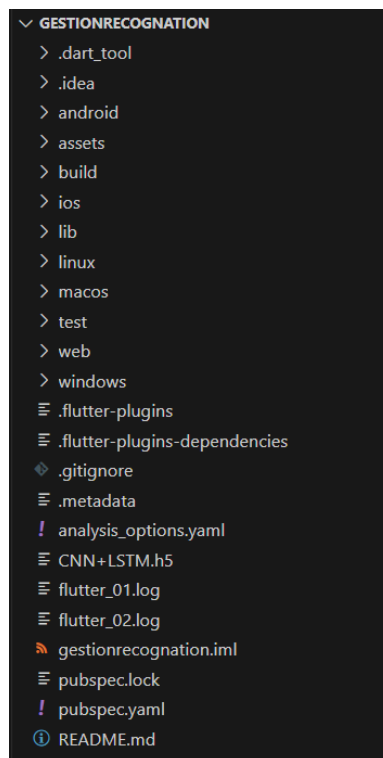


FIGURE 5.1 – Structure du Projet Flutter

5.1.1.2 Conception des Interfaces Utilisateur

La conception de l'interface utilisateur (UI) de Flutter est basée sur l'utilisation d'outils, qui sont les éléments constitutifs de l'application. Chaque élément de l'interface, que ce soit un bouton, une photo ou du texte, est représenté par un outil. Flutter offre une grande bibliothèque d'outils personnalisables, allant des composants de base comme **Container** et **Text** aux widgets complexes comme **ListView** et **GridView**.

Lors de la conception des interfaces d'applications de reconnaissance gestuelle, nous avons choisi une approche centrée sur l'utilisateur, en mettant l'accent sur la navigation simple et l'interaction fluide. Les écrans sont structurés de manière hiérarchique, avec une page principale de réception permettant d'accéder aux différentes fonctionnalités de l'application.

Les interfaces sont conçues pour être réactives, s'adaptant aux différentes tailles d'écran et orientations des appareils. Nous avons également intégré des animations et des transitions entre les écrans pour améliorer l'expérience utilisateur, en utilisant les widgets **AnimatedContainer** et **Hero**.

Pour tester et améliorer la conception de l'interface, nous avons utilisé des simulateurs Android et iOS. Ces simulateurs vous permettent de simuler différentes configurations matérielles, de tester l'interaction de l'application avec une précision différente et de vérifier le comportement de l'animation. Les simulateurs peuvent rapidement détecter et corriger les problèmes d'interface utilisateur avant de les déployer sur des appareils physiques, assurant ainsi une expérience optimale pour l'utilisateur sur un large éventail de périphériques.

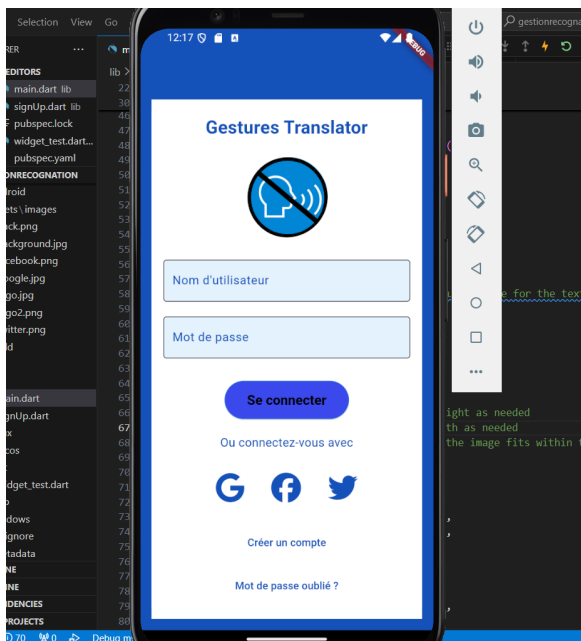


FIGURE 5.2 – Page d'accueil

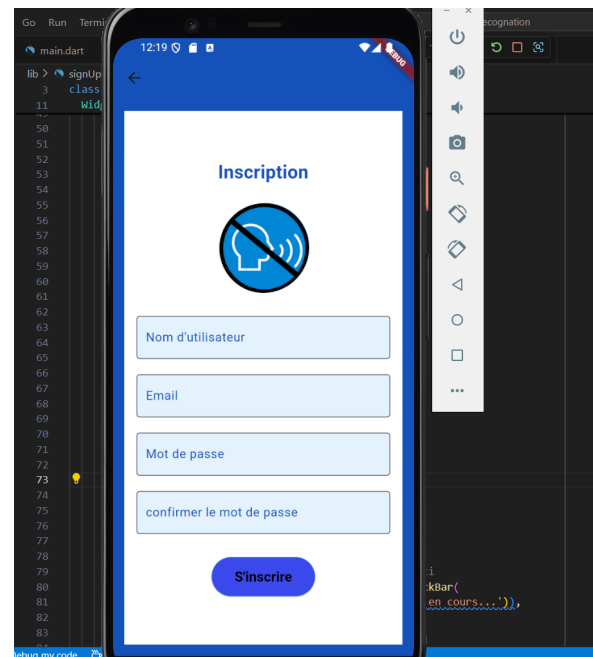


FIGURE 5.3 – Page d'inscription

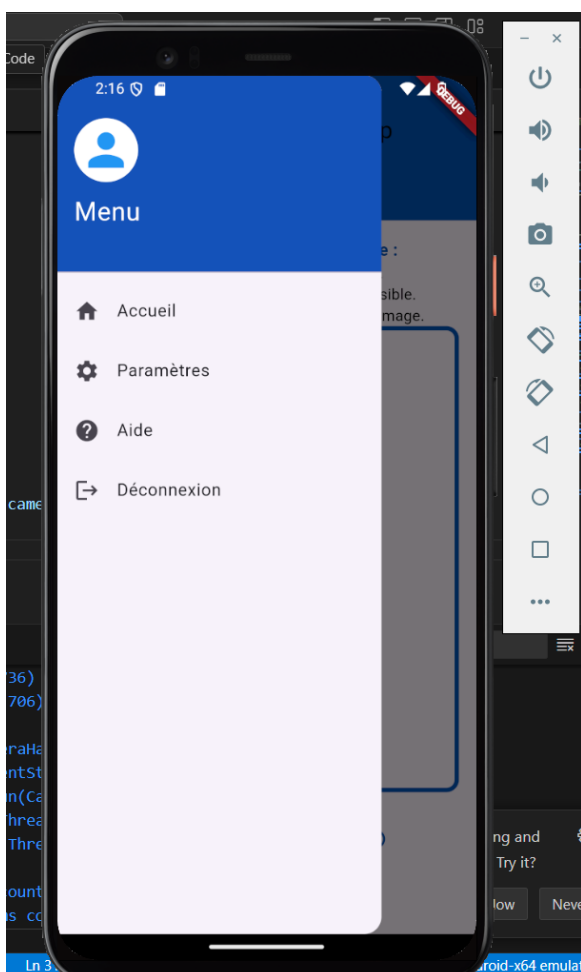


FIGURE 5.4 – Menu de Navigation

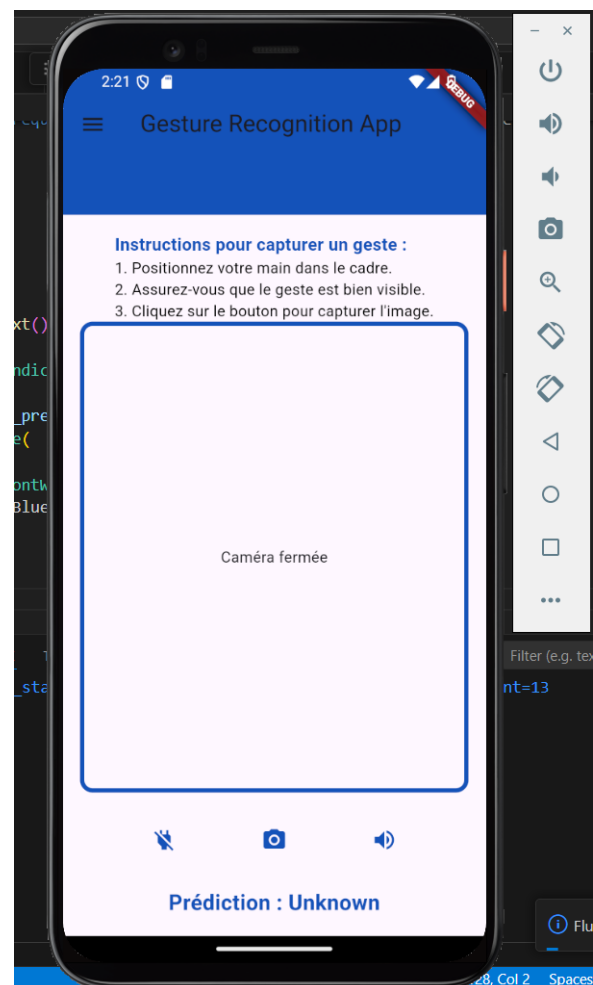


FIGURE 5.5 – Page Principale

5.1.1.3 Gestion des États et Navigation

La gestion des états dans l'application Flutter est un aspect clé pour assurer la réactivité et la rationalisation de l'interface utilisateur. Dans notre projet, nous avons utilisé le modèle **Provider** pour gérer les états de l'application. Ce modèle permet de séparer la logique métier de l'interface utilisateur, en maintenant un état global partagé entre les différents widgets.

La navigation entre les différentes pages de l'application est gérée par le système de routage de Flutter. Nous avons configuré des routes nommées pour chaque écran de l'application, facilitant ainsi la navigation et la gestion des transitions entre les pages. Le widget **Navigator** est utilisé pour pousser et popper des pages sur la pile de navigation, offrant une expérience utilisateur cohérente et intuitive.

Pour des scénarios plus complexes, comme la navigation conditionnelle ou les paramètres de défilement entre les pages, nous avons intégré des solutions personnalisées utilisant des reconnections et des moniteurs de navigation. Cette approche permet un contrôle précis du flux de navigation et garantit que les utilisateurs naviguent dans l'application de manière logique et fluide.

5.2 Développement Backend avec Flask

Dans cette section, nous explorons le développement de l'application de reconnaissance de gestes en utilisant Flask, un framework Python léger et puissant. Flask facilite la création d'API RESTful qui interagissent avec l'application Flutter côté client. Le backend gère le traitement des images, l'extraction des points clés des gestes capturés, et la prédiction des actions à l'aide d'un modèle pré-entraîné CNN + LSTM.

5.2.1 Structure de l'Application Flask

L'application Flask est conçue de manière modulaire pour séparer les responsabilités, facilitant ainsi la maintenance et l'évolution du code. La structure de l'application comprend les composants suivants :

- **Initialisation de l'Application** : Point d'entrée de l'application Flask, où sont démarrées l'application et les dépendances nécessaires comme MediaPipe et TensorFlow.
- **Routes** : Points d'accès à l'application via HTTP. La route principale est `/predict`, qui traite les requêtes POST contenant des images pour la prédiction des gestes.
- **Modèle Pré-entraîné** : Le modèle CNN+LSTM est chargé au démarrage de l'application pour être utilisé dans les prédictions.
- **Fonctions Utilitaires** : Incluent la détection de gestes avec MediaPipe et l'extraction des points clés pour la prédiction.

La structure des fichiers suit une convention standard avec le fichier principal `app.py` pour l'initialisation de l'application et les routes. Les modèles, configurations et utilitaires sont organisés en sous-dossiers pour une meilleure organisation.

5.2.2 Création des API pour la Communication avec Flutter

Les API Flask permettent la communication entre l'application Flutter et le backend. L'API principale, accessible via la route `/predict`, reçoit les images envoyées par Flutter et retourne la prédiction de l'action correspondante.

- **Réception des Fichiers** : L'API accepte les fichiers image envoyés via des requêtes POST, les convertit au format approprié pour le traitement.
- **Traitement de l'Image** : L'image reçue est redimensionnée et convertie pour être compatible avec le modèle MediaPipe.

- **Extraction des Points Clés** : MediaPipe extrait les points clés du visage, des mains et du corps. Ces points clés sont préparés pour le modèle de prédiction.
- **Prédiction et Réponse** : Les points clés sont passés au modèle CNN+LSTM, qui génère une prédiction. La prédiction est renvoyée au client Flutter sous forme de réponse JSON.

5.2.3 Chargement et Utilisation du Modèle CNN+LSTM

Le modèle CNN+LSTM est pré-entraîné et est chargé lors du démarrage de l'application Flask. Voici les principales étapes liées à son utilisation :

- **Chargement du Modèle** : Le modèle est chargé depuis un fichier `.h5` à l'aide de TensorFlow. Cette étape est cruciale pour préparer le modèle à la prédiction.
- **Préparation des Données** : Les points clés extraits par MediaPipe sont formatés pour correspondre aux dimensions d'entrée du modèle.
- **Prédiction** : Les données préparées sont passées au modèle pour générer une prédiction. La sortie est la probabilité pour chaque action, et l'action avec la probabilité la plus élevée est sélectionnée comme prédiction finale.

5.2.4 Gestion des Requêtes et Réponses

La gestion des requêtes et des réponses est essentielle pour une communication fluide entre le backend et le frontend :

- **Validation des Requêtes** : Le serveur vérifie si la requête contient toutes les informations nécessaires, comme le fichier image. En cas de données manquantes ou invalides, une réponse d'erreur est renvoyée.
- **Traitement des Réponses** : Après la prédiction, le backend génère une réponse JSON contenant le résultat. En cas d'erreur, un message d'erreur est également renvoyé en JSON.
- **Gestion des Erreurs** : Flask capture les erreurs et renvoie des messages détaillés en cas de problèmes lors du traitement des requêtes, facilitant le diagnostic et la résolution des problèmes.

5.2.5 Intégration du Modèle de Machine Learning

L'intégration d'un modèle de machine learning dans une application web permet de transformer les capacités du modèle en une fonctionnalité accessible via l'interface utilisateur. Nous détaillons ici l'intégration du modèle CNN + LSTM dans l'application Flask.

5.2.5.1 Importation du Modèle CNN + LSTM

Pour intégrer le modèle CNN + LSTM, nous chargeons le modèle sauvegardé au format `.h5` :

```
from flask import Flask, request, jsonify
import cv2
import numpy as np
import mediapipe as mp
import tensorflow as tf
from PIL import Image

app = Flask(__name__)

# Initialize MediaPipe Holistic and Drawing Utils
mp_holistic = mp.solutions.holistic
mp_drawing = mp.solutions.drawing_utils

# Load the pre-trained model
model = tf.keras.models.load_model(r'C:\Users\Nessrine\StudioProjects\flask-app\CNN+LSTM2.h5')
```

FIGURE 5.6 – Intégration du Modèle

Le modèle est chargé depuis le chemin spécifié et est prêt à être utilisé pour les prédictions. Il est chargé au démarrage de l'application pour éviter de recharger le modèle à chaque requête, ce qui améliore les performances.

5.2.5.2 Prétraitement des Données d'Entrée

Avant d'envoyer des données d'entrée au modèle, elles doivent être prétraitées. Nous utilisons MediaPipe pour détecter les articulations et les poses :

```
def mediapipe_detection(image):
    with mp_holistic.Holistic(static_image_mode=True) as holistic:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = holistic.process(image)
        return results

def extract_keypoints(results):
    pose = np.array([[res.x, res.y, res.z, res.visibility] for res in results.pose_landmarks.landmark]).flatten() if results.pose_landmarks else np.zeros(33*4)
    face = np.array([[res.x, res.y, res.z] for res in results.face_landmarks.landmark]).flatten() if results.face_landmarks else np.zeros(468*3)
    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten() if results.left_hand_landmarks else np.zeros(21*3)
    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten() if results.right_hand_landmarks else np.zeros(21*3)
    return np.concatenate([pose, face, lh, rh])
```

FIGURE 5.7 – Prétraitement des Données d'Entrée

L'image est traitée pour extraire les points clés nécessaires à la prédiction. Ces points sont formatés pour correspondre aux entrées attendues par le modèle CNN + LSTM.

5.2.5.3 Génération des Prédictions et Traitement des Résultats

Après le prétraitement, les données d'entrée sont utilisées pour générer une prédiction :

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        if 'file' not in request.files:
            return jsonify({'error': 'No file part in the request'}), 400

        file = request.files['file']
        if file.filename == '':
            return jsonify({'error': 'No selected file'}), 400

        image = Image.open(file.stream).convert('RGB')
        image = image.resize((224, 224))
        image_array = np.array(image)

        image_bgr = cv2.cvtColor(image_array, cv2.COLOR_RGB2BGR)
        results = mediapipe_detection(image_bgr)
        keypoints = extract_keypoints(results)

        if keypoints.shape[0] == 1662:
            keypoints = np.expand_dims(keypoints, axis=0)
            keypoints = np.expand_dims(keypoints, axis=1)
            keypoints = np.repeat(keypoints, 30, axis=1)
        else:
            return jsonify({'error': 'Invalid keypoints shape'}), 400

        prediction = model.predict(keypoints)
        predicted_class_index = np.argmax(prediction) # Get the index of the highest probability

        # Assuming you have a list of class names
        class_names = [ 'n3ass', 'lunettes', 'je froid', 'salam', 'Chokran', 'je t aime', 'La', 'j aime', 'j aime pas', 'Nakel', 'Nchrob', 'Parfait',
            'Telephone', 's il te plait', 'pleun', 'fort', 'lissar', 'Maqtach',
            'Limine', 'sghir', 'kbin', 'stop', 'sma3ni', 'fakar', 'chof' ]

        predicted_class = class_names[predicted_class_index] # Get the class name
        result = {'predicted_action': predicted_class}

        return jsonify(result)
    except Exception as e:
        print(f"Error: {e}")
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

FIGURE 5.8 – Génération des Prédictions

Le modèle génère une prédiction basée sur les données prétraitées. La prédiction, correspondant à l'action la plus probable, est renvoyée au client Flutter en réponse JSON. Cette réponse est utilisée par l'application Flutter pour afficher la prédiction à l'utilisateur.

Cette intégration permet de transformer le modèle d'apprentissage automatique en une fonctionnalité pratique pour l'utilisateur final, facilitant l'interaction avec l'application.

5.3 Connexion entre Flutter et Flask

5.3.1 Configuration des Requêtes HTTP dans Flutter

Pour établir la connexion entre l'application Flutter et le serveur Flask, nous devons configurer les requêtes HTTP dans Flutter. Cela implique l'utilisation de la bibliothèque `http` pour envoyer des requêtes HTTP POST contenant les images capturées par la caméra. Voici comment nous configurons les requêtes HTTP :

```
Future<void> _uploadImage(File image) async {
  setState(() {
    _isProcessing = true;
  });

  final request = http.MultipartRequest(
    'POST',
    Uri.parse('http://10.0.2.2:5000/predict'),
  );
  request.files.add(await http.MultipartFile.fromPath('file', image.path));

  final response = await request.send();

  if (response.statusCode == 200) {
    final responseData = await response.stream.toBytes();
    final responseJson = jsonDecode(utf8.decode(responseData));
    setState(() {
      _prediction = responseJson['predicted_action'].toString();
    });
  } else {
    setState(() {
      _prediction = 'Error: ${response.statusCode}';
    });
  }

  setState(() {
    _isProcessing = false;
  });
}
```

FIGURE 5.9 – Configuration des Requêtes HTTP

Dans ce code, nous créons une requête `MultipartRequest` pour envoyer une image au serveur Flask via la requête POST. Ensuite, nous vérifions la réponse du serveur pour la prédiction.

5.3.2 Traitement des Réponses de l'API Flask dans Flutter

Après l'envoi de la demande HTTP, nous devons traiter la réponse de l'API Flask. Le serveur Flask réintègre les prédictions dans le modèle JSON, que nous devons décoder pour obtenir l'action attendue.

```
if (response.statusCode == 200) {
  final responseData = await response.stream.toBytes();
  final responseJson = jsonDecode(utf8.decode(responseData));
  setState(() {
    _prediction = responseJson['predicted_action'].toString();
  });
} else {
  setState(() {
    _prediction = 'Error: ${response.statusCode}';
  });
}

setState(() {
  _isProcessing = false;
});
}
```

FIGURE 5.10 – Traitement des Réponses de l'API

Ici, nous utilisons la fonction `jsonDecode` pour convertir les données JSON en objet Dart. Nous mettons ensuite à jour le statut de l'ordre avec la prévision obtenue.

5.3.3 Traduction en message vocal

```
void _initializeTts() async {
  await flutterTts.setLanguage('fr-FR');
  await flutterTts.setSpeechRate(0.5);
}

void _translateToVoice() async {
  if (_prediction.isNotEmpty) {
    await flutterTts.speak(_prediction);
  }
}
```

FIGURE 5.11 – Traduction en Message Vocal

La fonction `_initializeTts()` configure le système de synthèse vocale en définissant la langue à `fr-FR` (Français) et en ajustant le taux de parole à une vitesse modérée. Ce réglage est essentiel pour garantir que le texte soit prononcé correctement et clairement. Ensuite, la fonction `_translateToVoice()` utilise cette configuration pour convertir le texte de la prédiction en

discours. Lorsque `_prediction` contient du texte, celui-ci est prononcé par le système de synthèse vocale, offrant ainsi une rétroaction audible des résultats de la prédiction. Ensemble, ces fonctions permettent d'améliorer l'interaction utilisateur en fournissant des réponses vocales adaptées dans l'application.

5.3.4 Gestion des Erreurs et Débogage

La gestion des erreurs est essentielle pour assurer une expérience utilisateur transparente. Nous devons gérer les erreurs potentielles lors de la prise de photos, de l'envoi de demandes et de la réception de réponses.

```
Future<void> _takePicture() async {
  try {
    await _initializeControllerFuture;
    final image = await _controller.takePicture();

    final tempDir = await getTemporaryDirectory();
    final tempPath = '${tempDir.path}/temp_image.jpg';
    await image.saveTo(tempPath);

    final file = File(tempPath);
    await _uploadImage(file);
  } catch (e) {
    print('Error taking picture: $e');
    setState(() {
      _prediction = 'Error: ${e.toString()}';
    });
  }
}
```

FIGURE 5.12 – Gestion des Erreurs et Débogage

Le bloc `try-catch` est utilisé pour capturer et afficher les erreurs éventuelles lors de la prise de photo ou de l'envoi de la requête. Les messages d'erreur sont affichés à l'utilisateur pour fournir des informations utiles sur les problèmes rencontrés.

```
1/1 — 0s 65ms/step
INFO:werkzeug:127.0.0.1 - - [06/Sep/2024 15:02:18] "POST /predict HTTP/1.1" 200 -
```

FIGURE 5.13 – Prediction avec succès

Ce message dans le terminal de Flask indique que la connexion entre Flask et Flutter a été établie avec succès, et que la prédiction a été effectuée correctement.

5.4 Tests Fonctionnels

5.4.1 Tests Fonctionnels de l'Application Mobile

Les tests fonctionnels assurent que l'application mobile répond aux exigences et fonctionne comme prévu. Les principales étapes sont :

1. **Test de Capture d'Image** : Vérification du bon fonctionnement de la caméra pour capturer des images.
2. **Test d'Envoi d'Image** : S'assurer que les images sont correctement envoyées au serveur Flask via HTTP POST.
3. **Test de Réception de Prédictions** : Vérification de l'affichage correct des prédictions reçues du serveur.
4. **Test de la Synthèse Vocale** : Tester la conversion des prédictions en voix de manière précise et intelligible.

Ces tests sont réalisés manuellement et automatiquement pour garantir la couverture complète des fonctionnalités.

5.4.2 Tests de l'API Flask et du Modèle

Les tests de l'API Flask et du modèle visent à garantir le bon fonctionnement du serveur et la précision du modèle. Les étapes incluent :

1. **Test de la Route API** : Vérification de l'accessibilité et de la réponse de la route `/predict` aux requêtes POST avec fichiers image.
2. **Test de Traitement des Images** : S'assurer que le serveur traite correctement les images et renvoie les prédictions attendues.
3. **Test de la Précision du Modèle** : Évaluation de la précision du modèle en comparant les prédictions aux résultats attendus sur un ensemble de test.
4. **Test des Performances** : Mesurer le temps de réponse de l'API pour garantir une bonne expérience utilisateur.

Ces tests assurent la fonctionnalité de l'API et la performance du modèle de prédiction.

5.4.3 Validation de l'Intégration et Performances

La validation assure le bon fonctionnement combiné de tous les composants du système et la conformité aux attentes de performance. Les étapes principales sont :

1. **Test d'Intégration** : Vérification du fonctionnement fluide entre l'application mobile, l'API Flask et le modèle.
2. **Test de Performance de l'Application** : Évaluation des performances de l'application, incluant le temps de réponse et la fluidité de l'interface.
3. **Test de Scalabilité** : Test sous différentes charges de travail pour vérifier la capacité à gérer une augmentation des utilisateurs ou des requêtes.
4. **Validation des Exigences** : Vérification que le système répond aux exigences spécifiées en termes de fonctionnalités et de performances.

La validation garantit que le système est fiable et performant.

5.5 Déploiement et Maintenance

5.5.1 Déploiement de l'Application Flutter

Le déploiement de l'application Flutter comporte les étapes suivantes :

1. **Préparation pour la Publication** : Configuration pour la publication incluant la mise à jour des informations de version et des paramètres de build.
2. **Génération des Builds** : Création des builds pour Android (APK ou AAB) et iOS (IPA) en utilisant Flutter.
3. **Test sur des Appareils Réels** : Vérification du bon fonctionnement de l'application sur des appareils réels.
4. **Publication sur les Stores** : Soumission de l'application aux stores (Google Play Store et App Store), création des fiches descriptives, et configuration des options de publication.
5. **Surveillance et Feedback** : Suivi des performances et collecte des avis utilisateurs pour identifier et résoudre les problèmes.

5.5.2 Déploiement du Backend Flask

Le déploiement du backend Flask implique les étapes suivantes :

1. **Configuration de l'Environnement** : Installation des dépendances nécessaires avec `pip`.
2. **Configuration du Serveur** : Choix et configuration du serveur web (Nginx, Apache, AWS, Heroku, Azure).
3. **Déploiement de l'Application** : Transfert des fichiers et configuration du serveur pour exécuter l'application Flask.
4. **Tests Post-Déploiement** : Vérification du bon fonctionnement de l'application Flask sur le serveur.
5. **Surveillance et Maintenance** : Mise en place d'outils de surveillance et gestion régulière des mises à jour et problèmes.

5.5.3 Stratégies de Maintenance et Mise à Jour

Pour assurer la pérennité et la performance continue du système :

1. **Planification des Mises à Jour** : Préparation des mises à jour régulières pour sécurité, nouvelles fonctionnalités, et améliorations.
2. **Gestion des Bugs** : Mise en place d'un système de signalement et correction des bugs.
3. **Surveillance Continue** : Utilisation d'outils pour suivre la performance et résoudre les problèmes.
4. **Sauvegardes Régulières** : Réalisation de sauvegardes des données et configurations pour éviter les pertes.
5. **Communication avec les Utilisateurs** : Information des utilisateurs sur les mises à jour et problèmes connus.

Ces stratégies garantissent que l'application et le backend restent fiables, sécurisés et performants.

5.6 Conclusion

5.6.1 Résumé du Fonctionnement de l'Application

Cette section présente un aperçu global du fonctionnement de l'application mobile développée. Nous décrivons les principales fonctionnalités, l'intégration du backend Flask et du modèle de machine learning, ainsi que l'interaction utilisateur.

1. Page d'Accueil :

- **Champs de Saisie** : Nom d'utilisateur et mot de passe.
- **Bouton de Connexion** : Pour accéder au compte.
- **Options de Connexion Sociale** : Facebook, Google, Twitter.
- **Lien pour Créer un Compte** : Inscription pour nouveaux utilisateurs.
- **Lien de Réinitialisation du Mot de Passe** : Pour récupérer un mot de passe oublié.



FIGURE 5.14 – Page d'Accueil

2. Page d'Inscription :

- **Nom d'Utilisateur** : Saisie d'un nom unique.
- **Adresse E-mail** : Pour la communication et la récupération de mot de passe.
- **Mot de Passe** : Saisie d'un mot de passe sécurisé.
- **Confirmation du Mot de Passe** : Saisie pour confirmation.

The image shows a mobile application interface for a registration page. At the top, there is a blue header bar with a back arrow on the left and a red 'Auto' button on the right. Below the header, the title 'Inscription' is displayed in blue. Underneath the title is a circular icon with a blue background, a white outline of a head with a brain, and a black diagonal line crossing it out. Below the icon are four light blue rectangular input fields with rounded corners, each containing a label: 'Nom d'utilisateur', 'Email', 'Mot de passe', and 'confirmer le mot de passe'. At the bottom of the form is a blue rounded rectangular button with the text 'S'inscrire' in white.

FIGURE 5.15 – Page d'Inscription

3. Page de Reconnaissance Gestuelle :

- **Caméra Centrale** : Capture des gestes en temps réel.
- **Boutons de Contrôle** :
 - 1- Activer/Désactiver la caméra.
 - 2- Prédire le geste.
 - 3- Traduire en message vocal.
- **Conseils pour une Prédiction Optimale** : Éclairage, distance, stabilité du geste.

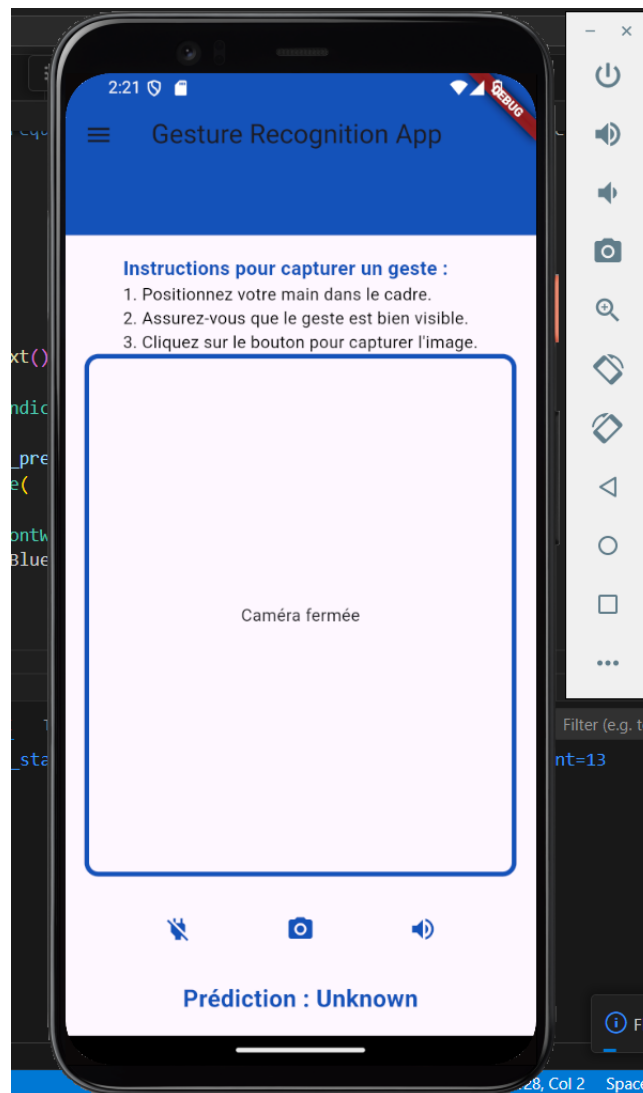


FIGURE 5.16 – Page de Reconnaissance Gestuelle

4. Menu de Navigation :

- **Paramètres** : Personnalisation des préférences.
- **Profil** : Consultation et modification des informations personnelles.
- **Accueil** : Retour à la page d'accueil.
- **Aide** : Accès aux ressources de support.

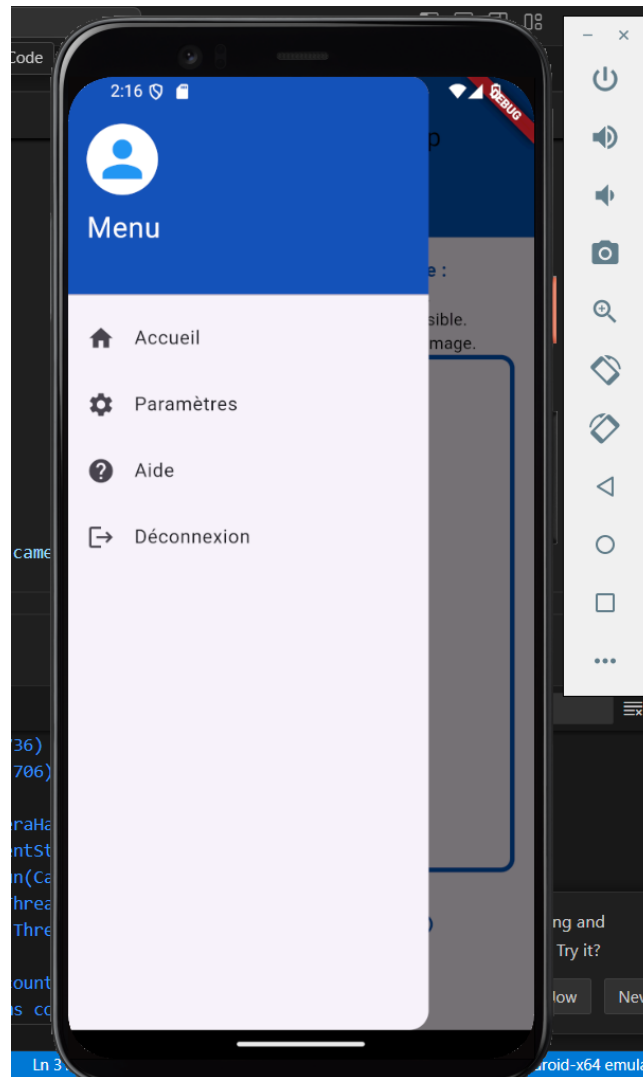


FIGURE 5.17 – Menu de Navigation

5.6.2 Évaluation des Résultats et Performances

Nous analysons ici les résultats des tests fonctionnels et de performance. Nous discutons de la précision du modèle CNN + LSTM, de la réactivité de l'application et de l'intégration Flutter-Flask. Les résultats des tests, sont présentés ci-dessous 11 gestes parmi les 30 gestes effectuées.

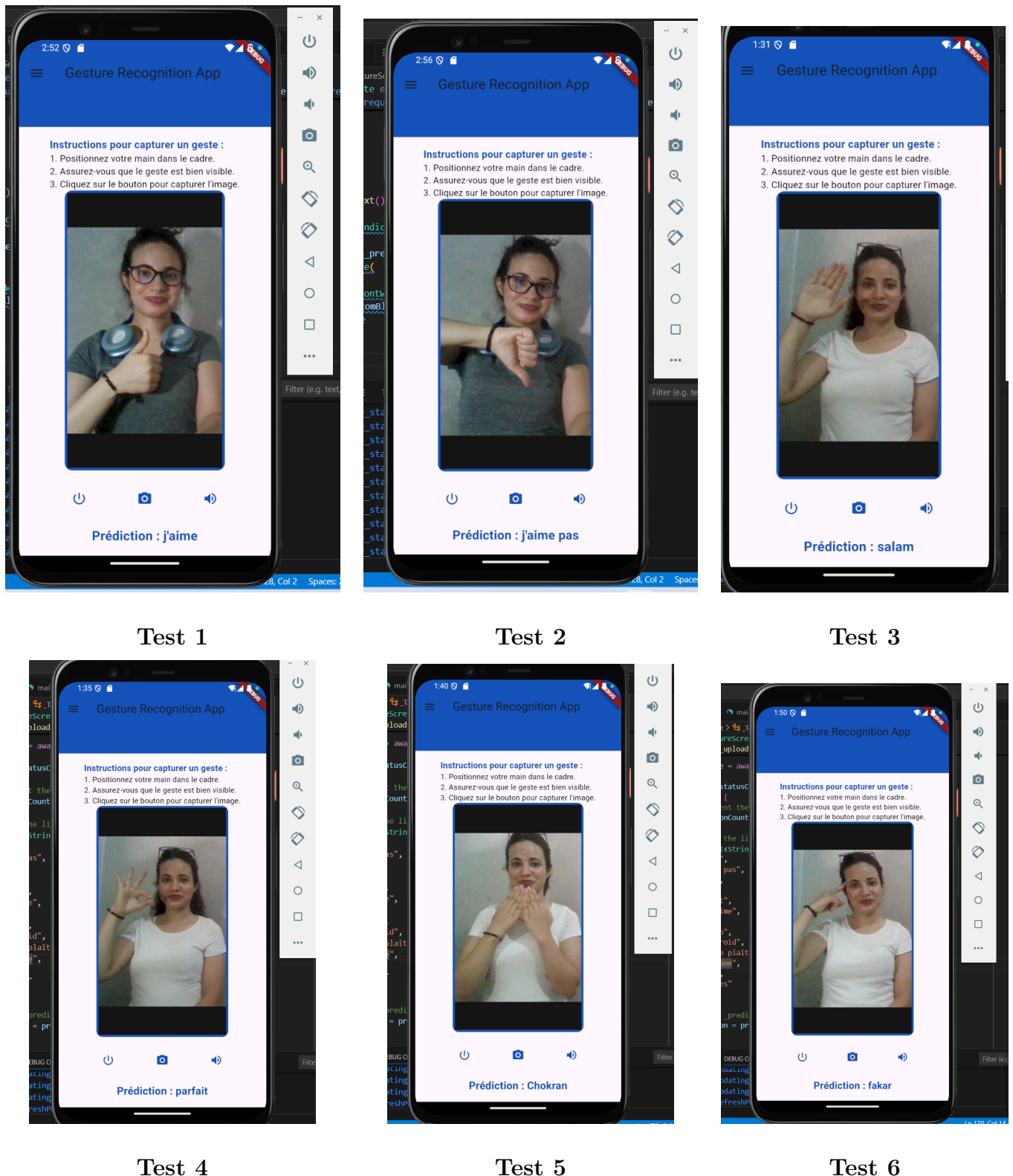
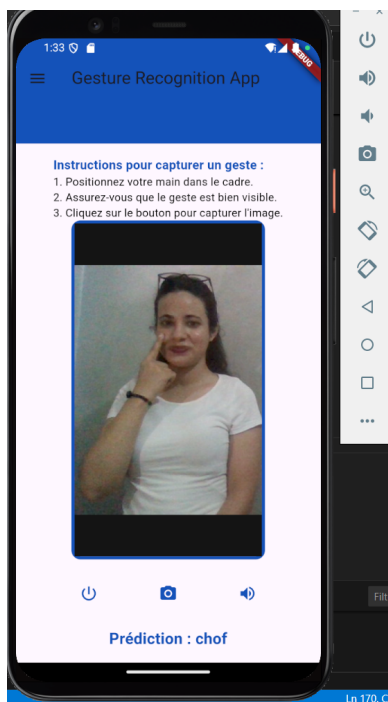
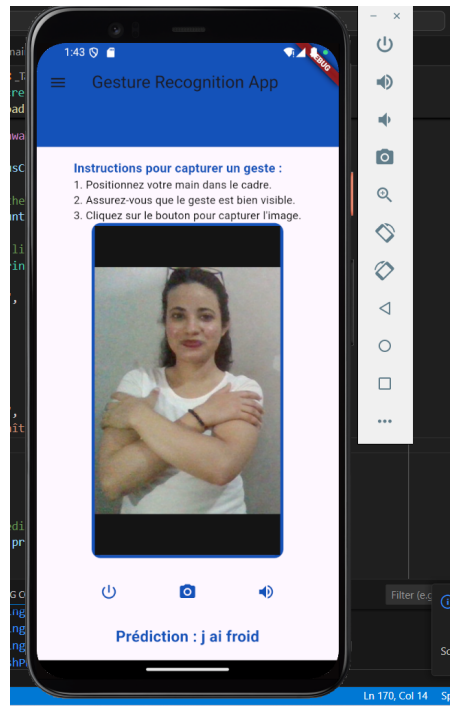


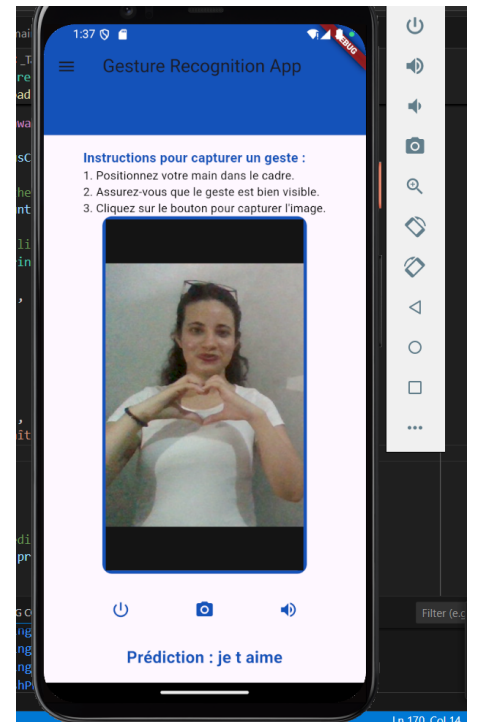
FIGURE 5.18 – Test des gestes



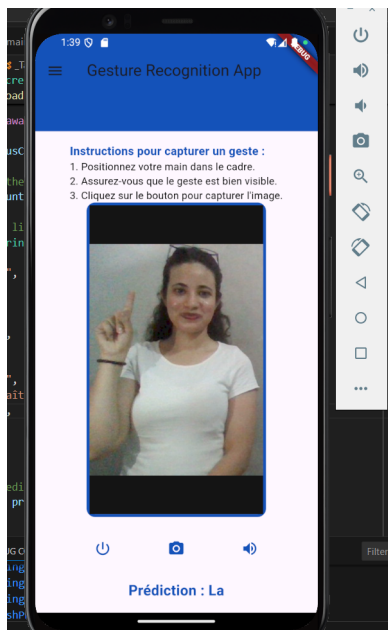
Test 7



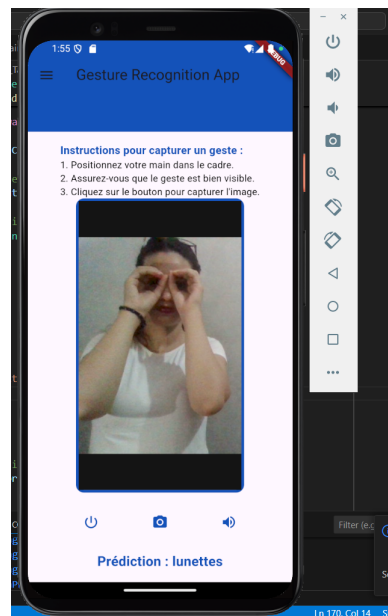
Test 8



Test 9



Test 10



Test 11

FIGURE 5.19 – Test des gestes

5.6.3 Perspectives et Améliorations Futures

Dans le cadre du projet, plusieurs perspectives et améliorations peuvent être envisagées pour optimiser les résultats et étendre les capacités du système. Ces pistes permettent de consolider les acquis tout en ouvrant de nouvelles opportunités de développement.

- **Amélioration des algorithmes de reconnaissance** : Une première perspective pourrait consister à explorer d'autres modèles d'apprentissage automatique ou profond, tels que les réseaux de neurones convolutifs 3D ou les réseaux de neurones récurrents avec des mécanismes d'attention. Ces modèles pourraient améliorer la précision et la robustesse de la reconnaissance des gestes, en prenant mieux en compte la continuité temporelle et les variations interpersonnelles.
- **Enrichissement du dataset** : Pour rendre le modèle plus performant et généralisable, il serait pertinent d'enrichir le dataset en capturant davantage de gestes et en incluant des variations dans les conditions de capture (différents angles de caméra, luminosités variées, etc.). L'inclusion de gestes plus complexes pourrait également renforcer les capacités de détection du système.
- **Optimisation des performances en temps réel** : L'une des améliorations clés serait de réduire les temps de latence pour une reconnaissance plus rapide des gestes en temps réel. Cela pourrait être réalisé par une meilleure optimisation du code ou par l'utilisation de dispositifs matériels spécialisés, tels que des processeurs graphiques (GPU) ou des unités de traitement des tenseurs (TPU).
- **Extension à d'autres types de gestes ou de langues des signes** : Le système pourrait être étendu pour reconnaître des gestes provenant d'autres langues des signes, telles que la Langue des Signes Française (LSF) ou la Langue des Signes Américaine (ASL). Cela ouvrirait de nouvelles possibilités d'application dans des contextes multilingues ou multiculturels.
- **Intégration de la traduction en plusieurs langues** : Une amélioration majeure serait de permettre la traduction des gestes non seulement en texte, mais aussi en plusieurs langues, telles que l'anglais, le français, l'arabe, etc. Cela permettrait de rendre le système plus accessible à un public mondial et de faciliter la communication entre des personnes parlant différentes langues, notamment dans des situations internationales ou dans des contextes d'apprentissage de la langue des signes.
- **Amélioration de l'interface utilisateur** : Pour faciliter l'utilisation de l'application par des non-experts, il serait bénéfique de développer une interface utilisateur plus intuitive et ergonomique.

Conclusion générale

Ce projet représente une étape importante dans le développement d'une application mobile innovante dédiée à la reconnaissance gestuelle, intégrant des techniques de machine learning et offrant une interface utilisateur intuitive grâce à Flutter. En combinant le modèle d'apprentissage profond basé sur l'architecture CNN + LSTM avec une interface de fond développée dans Flask, nous avons réussi à créer une solution capable de traduire des gestes en messages texte et audio, rendre la communication plus accessible pour les personnes ayant une déficience auditive ou de la parole.

Le processus de développement comporte de nombreux défis techniques, notamment l'intégration transparente du modèle d'apprentissage automatique avec l'application mobile, la gestion des communications entre les différents composants de l'application et la garantie d'une expérience utilisateur optimale. Grâce à une série de tests fonctionnels et de validation, nous avons pu assurer la durabilité et la fiabilité de l'application.

Le déploiement des applications, ainsi que la mise en place de stratégies de maintenance, assurent leur pérennité et leur évolutivité. Les utilisateurs peuvent désormais interagir avec l'application de manière intuitive, en utilisant des gestes simples pour obtenir des traductions en temps réel.

En conclusion, ce projet démontre la puissance des technologies modernes lorsqu'elles sont bien intégrées pour résoudre les problèmes du monde réel. Il ouvre la voie à de futures améliorations et ajouts, tels que l'ajout de nouveaux gestes, l'amélioration des modèles de reconnaissance ou l'inclusion de fonctionnalités supplémentaires pour rendre l'application plus utile pour ses utilisateurs. Ces travaux fournissent une base solide pour les développements et innovations futurs en matière de technologies d'assistance.

Bibliographie et Webographie

Bibliographie

Grinberg, M. (2018). *Flask Web Development : Developing Web Applications with Python*. O'Reilly Media.

Google. (2022). *Flutter Documentation*. Google Developers.

Karpathy, A., Johnson, J., & Li, F. F. (2015). *Visualizing and Understanding Recurrent Networks*. In Proceedings of the International Conference on Learning Representations (ICLR).

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Palladino, L. (2021). *Building REST APIs with Flask*. Packt Publishing.

Murphy, K. P. (2012). *Machine Learning : A Probabilistic Perspective*. MIT Press.

Zeng, W., Wang, J., & Wang, Y. (2022). *A Survey on Deep Learning-Based Gesture Recognition*. IEEE Transactions on Neural Networks and Learning Systems, 33(5), 1925-1942.

Webographie

- Flutter Documentation. (2024). Récupéré de <https://docs.flutter.dev>
- Firebase Authentication Documentation. (2024). Récupéré de <https://firebase.google.com/docs/auth>
- Flask GitHub Repository. (2024). Récupéré de <https://github.com/pallets/flask>
- Flutter Development Q&A. (2024). Récupéré de <https://stackoverflow.com/questions/tagged/flutter>